

Diseño y modelado de bases de datos

Breve descripción:

Este componente aborda los fundamentos necesarios para comprender, estructurar y gestionar sistemas de información, iniciando con conceptos básicos de lógica y clasificación de bases de datos. Profundiza en los principios de las bases de datos relacionales, su diseño mediante modelos conceptual, lógico y físico, así como la aplicación de procesos como la normalización y definición de claves.

Mayo 2026

Tabla de contenido

Introducción	4
1. Fundamentos de lógica y algoritmia	6
1.1. Formatos de representación de la información.....	9
1.2. Clasificación de las bases de datos.....	14
1.3. Usos, diferencias y ventajas de los distintos tipos de bases de datos....	21
2. Fundamentos de bases de datos relacionales	26
2.1. Teoría de bases de datos relacionales	29
2.2. Componentes de una base de datos	34
2.3. Sistema gestor de bases de datos	38
2.4. Modelos de diseño de bases de datos	43
3. Diseño de bases de datos relacionales.....	51
3.1. Normalización de bases de datos.....	52
3.2. Modelo relacional.....	55
3.3. Tipos de claves.....	56
3.4. Entidades fuertes y débiles	58
3.5. Campos relacionales y dependencia de existencia.....	62
4. Lenguajes de sistemas gestores de bases de datos (DBMS)	65
4.1. Lenguaje de definición de datos (DDL).....	65

4.2.	Lenguaje de manipulación de datos (DML)	70
4.3.	Lenguaje de control de datos (DCL)	72
4.4.	Lenguaje de definición de vistas (VDL)	73
5.	Bases de datos NoSQL y su diseño	77
5.1.	Conceptos, principios y terminología NoSQL	77
5.2.	Modelos de bases de datos NoSQL	80
5.3.	Tipos de bases de datos NoSQL	81
5.4.	Arquitecturas y esquemas de datos	83
5.5.	Diseño de bases de datos NoSQL	84
	Síntesis	87
	Glosario	89
	Referencias bibliográficas	91
	Créditos	92

Introducción

Este componente formativo se orienta al desarrollo de competencias fundamentales para la estructuración, organización y gestión eficiente de la información en entornos digitales. En un contexto donde los datos son un activo estratégico, este documento permite comprender cómo diseñar soluciones que garanticen integridad, disponibilidad y coherencia en el manejo de la información.

El contenido inicia con los fundamentos de lógica y representación de la información, así como la clasificación de los diferentes tipos de bases de datos, incluyendo enfoques relacionales y no relacionales. Esto permite identificar las características, usos y ventajas de cada modelo según las necesidades del entorno.

Posteriormente, se abordan los principios de las bases de datos relacionales, profundizando en su teoría, componentes y en el uso de sistemas gestores de bases de datos. Se introduce el proceso de diseño mediante los modelos conceptual, lógico y físico, permitiendo estructurar soluciones de manera organizada y escalable.

El componente continúa con el diseño de bases de datos relacionales, incluyendo conceptos clave como normalización, modelo relacional, tipos de claves, entidades y dependencias, los cuales son esenciales para garantizar la integridad y eficiencia de los datos.

Asimismo, se incorporan los lenguajes propios de los sistemas gestores de bases de datos (DDL, DML, DCL y VDL), que permiten definir, manipular, controlar y visualizar la información dentro de los sistemas.

Finalmente, se introduce el enfoque de bases de datos NoSQL, abordando sus conceptos, modelos, tipos y arquitecturas, así como las estrategias de diseño basadas en patrones de acceso y análisis de relaciones. Esto brinda una visión actualizada y flexible frente a las necesidades de almacenamiento de grandes volúmenes de datos y aplicaciones modernas.

1. Fundamentos de lógica y algoritmia

Constituyen la base esencial para el diseño, análisis y desarrollo de soluciones informáticas, especialmente en el diseño y modelado de bases de datos. Este enfoque permite comprender cómo estructurar problemas, representar información y construir soluciones de manera ordenada, coherente y eficiente.

En el ámbito de los sistemas de información, la lógica y la algoritmia no solo permiten resolver problemas computacionales, sino que también facilitan la organización de datos, la definición de procesos y la toma de decisiones estructuradas, aspectos fundamentales en el diseño de bases de datos.

Este apartado aborda los principios básicos relacionados con la representación de la información, la clasificación de las bases de datos y el análisis de sus usos, diferencias y ventajas, proporcionando una visión integral del manejo de datos en diferentes entornos tecnológicos.

A. Importancia de la lógica y algoritmia en bases de datos

Su importancia radica en:

- **Lógica:** permite estructurar el pensamiento de manera ordenada.
- **Algoritmia:** facilita la definición de pasos necesarios para resolver un problema.

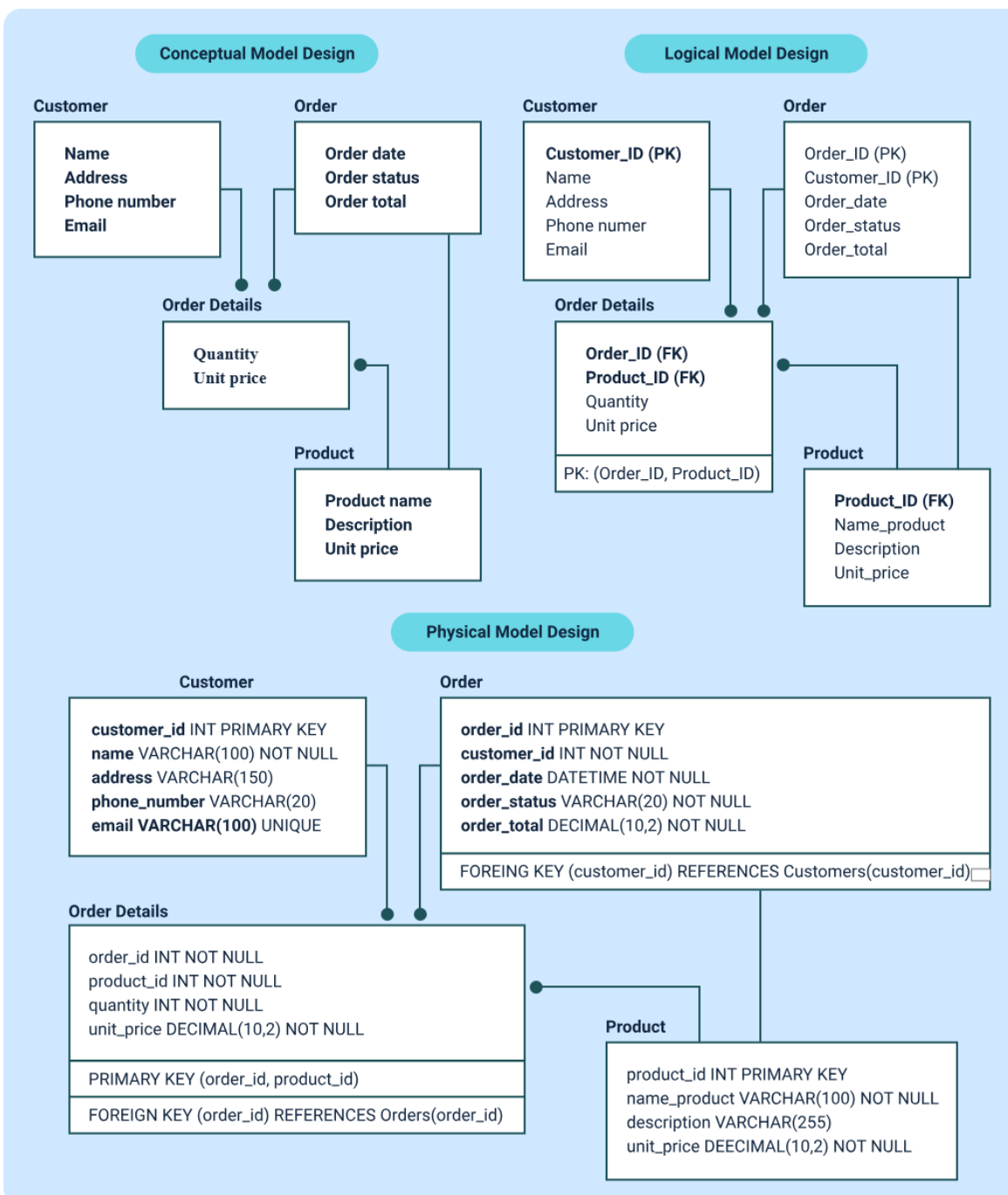
En el contexto de bases de datos, estos fundamentos permiten:

- Organizar la información de manera estructurada.
- Definir relaciones entre datos.
- Diseñar esquemas eficientes.

- Garantizar integridad y consistencia de la información.
- Optimizar procesos de consulta y almacenamiento.

La siguiente imagen da una relación explícita al respecto entre estos procesos:

Figura 1. Relación entre lógica, algoritmia y bases de datos



B. Componentes fundamentales de la lógica y algoritmia

Para comprender la lógica y la algoritmia, es necesario identificar los elementos que las conforman y su función dentro del proceso de resolución de problemas. A continuación, se presentan los componentes fundamentales y su descripción general:

Tabla 1. Componentes de la lógica y algoritmia

Componente	Descripción	Ejemplo
Lógica	Permite estructurar el pensamiento y establecer relaciones coherentes.	Evaluar una condición si-entonces para decidir si un estudiante aprueba o no una asignatura.
Algoritmo	Secuencia de pasos para resolver un problema.	Pasos ordenados para calcular el promedio de varias calificaciones.
Datos	Información que se procesa.	Lista de notas registradas en un sistema académico.
Procesos	Transformación de datos en resultados.	Operaciones matemáticas que calculan el promedio a partir de las notas.
Resultados	Salida generada por el sistema.	Promedio final mostrado en pantalla o almacenado en el sistema.

La lógica y la algoritmia son fundamentales para:

- Definir estructuras de almacenamiento de datos.

- Identificar relaciones entre entidades.
- Evitar redundancia e inconsistencias.
- Diseñar modelos eficientes.
- Optimizar consultas.

C. Características de una solución basada en lógica y algoritmia

A continuación, se presentan las principales características que orientan el desarrollo de soluciones lógicas y eficientes:

- Claridad en la definición del problema.
- Estructuración ordenada de pasos.
- Coherencia en el procesamiento de datos.
- Capacidad de adaptación a diferentes escenarios.
- Optimización de recursos.

El siguiente es el proceso lógico en la solución de problemas:

- Comprender el problema
- Realizar un plan
- Ejecutar el plan
- Analizar la solución

1.1. Formatos de representación de la información

La representación de los datos es un aspecto fundamental en los sistemas de información y en el diseño de bases de datos, ya que permite organizarlos y estructurarlos de forma comprensible y funcional. Los distintos formatos de

representación definen el modelado de los datos y facilitan su análisis, almacenamiento y procesamiento dentro de un sistema.

En el contexto del diseño y modelado de bases de datos, la representación de la información permite transformar datos del mundo real en estructuras formales que pueden ser interpretadas por sistemas computacionales. Esto es esencial para garantizar que la información sea consistente, accesible y útil para la toma de decisiones.

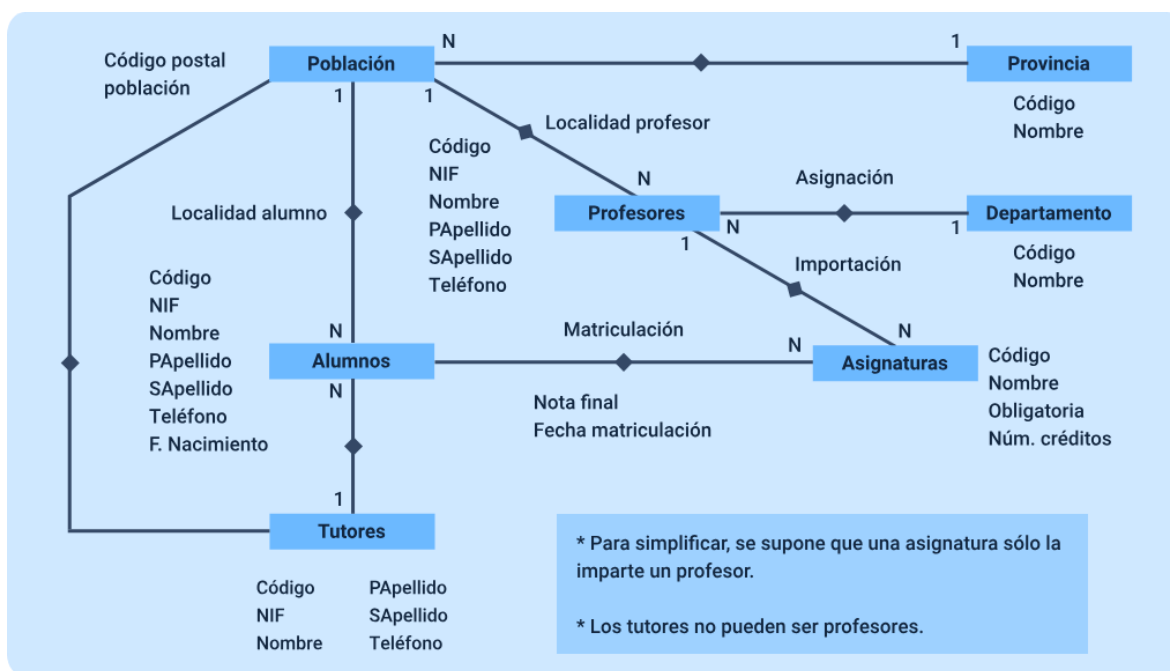
A. Importancia de la representación de la información

La adecuada representación de la información permite:

- Comprender la estructura de los datos.
- Facilitar el diseño de sistemas de información.
- Mejorar la organización y almacenamiento de datos.
- Reducir errores e inconsistencias.
- Optimizar el acceso y procesamiento de la información.

A continuación, se detalla un ejemplo sobre el proceso de la información:

Figura 2. Proceso de representación de la información



B. Tipos de formatos de representación de la información

La información puede representarse en diferentes formatos dependiendo del contexto, el propósito y el tipo de sistema en el que se utilice:

Tabla 2. Tipos de formatos de representación de la información

Formato	Descripción	Ejemplo
Textual	Representación mediante palabras o caracteres.	Documentos, registros.

Formato	Descripción	Ejemplo
Numérico	Representación mediante números.	Edad, salario.
Gráfico	Representación visual de la información.	Diagramas, gráficos.
Tabular	Organización en filas y columnas.	Tablas de datos.
Estructurado	Organización con reglas definidas.	Bases de datos.
Multimedia	Uso de imágenes, audio o video.	Archivos digitales.

En el diseño de bases de datos, la información se representa en tres niveles principales:

- **Conceptual**

Describe la información del mundo real a través de entidades, atributos y relaciones, permitiendo comprender el problema sin considerar aspectos técnicos. Facilita la comunicación entre usuarios y diseñadores del sistema.

- **Lógico**

Establece la estructura de los datos de manera formal, definiendo tablas, campos y relaciones, sin tener en cuenta aún el tipo de gestor de bases de datos o los detalles de almacenamiento.

- **Físico**

Especifica cómo se almacenan realmente los datos en el sistema, incluyendo archivos, índices, métodos de acceso y consideraciones de rendimiento según la tecnología utilizada.

Igualmente, para representar la información de manera estructurada, se utilizan diferentes modelos:

- **Modelo entidad-relación (ER):** representa entidades, atributos y relaciones.
- **Modelo relacional:** organiza la información en tablas.
- **Modelo orientado a objetos:** representa datos como objetos.
- **Modelo NoSQL:** permite estructuras flexibles para grandes volúmenes de datos.

Ejemplo aplicado

Supóngase un sistema de estudiantes:

Nombre: Juan

Edad: 20

Curso: Programación

Representación:

ID_Estudiante	Nombre	Edad	Curso
---------------	--------	------	-------

1	Juan	20	Programación
---	------	----	--------------

Modelo de base de datos

- Entidad: Estudiante
- Atributos: ID_Estudiante, Nombre, Edad, Curso

Los siguientes son los errores comunes en la representación de la información, por ello es importante tenerlos en cuenta para no cometerlos:

- Redundancia de datos.
- Falta de normalización.
- Ambigüedad en la definición de atributos.
- Mala estructuración de tablas.
- Inconsistencias en los datos.

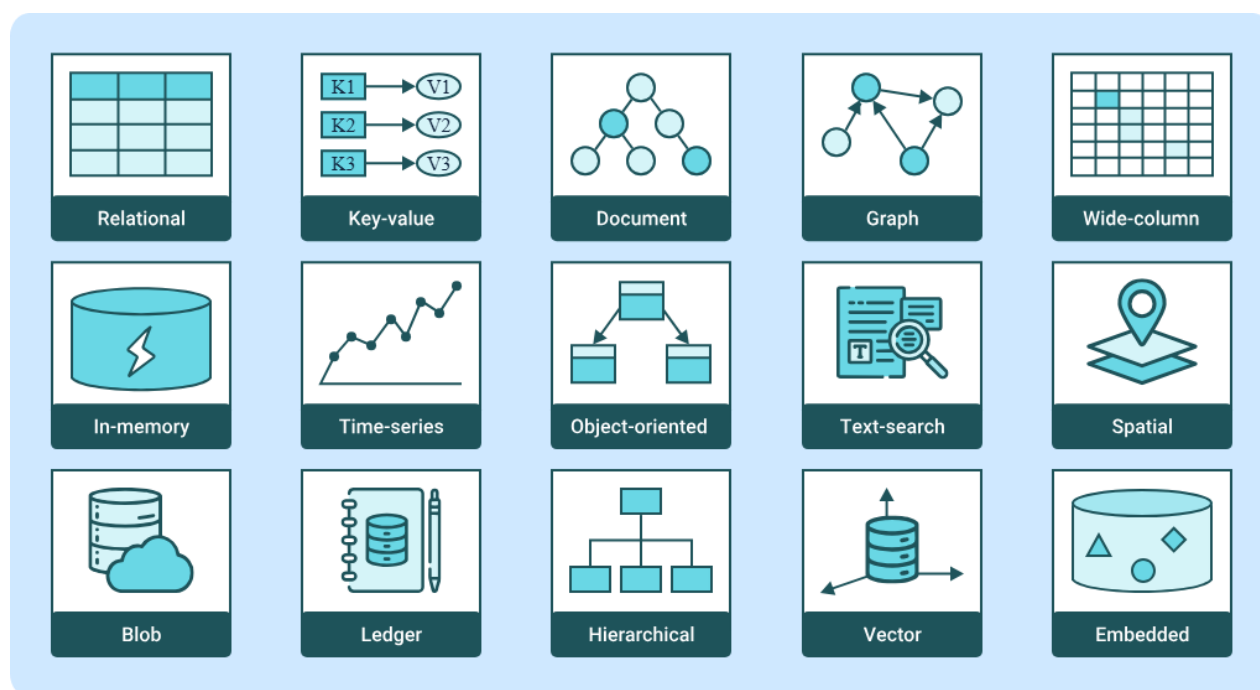
1.2. Clasificación de las bases de datos

La clasificación de las bases de datos responde a la necesidad de organizar, almacenar y gestionar la información de acuerdo con diferentes modelos, arquitecturas y requerimientos tecnológicos. A medida que evolucionan los sistemas de información, han surgido distintos tipos de bases de datos que permiten adaptarse a escenarios específicos, como grandes volúmenes de datos, alta velocidad de procesamiento o flexibilidad en la estructura de la información.

En este contexto, las bases de datos se clasifican principalmente en relacionales, NoSQL, in memory y NewSQL, cada una con características particulares que determinan su uso, ventajas y limitaciones. Esta clasificación permite seleccionar el tipo de base de datos más adecuado, según las necesidades del sistema, el tipo de información a manejar y el entorno tecnológico.

Aunque el enfoque principal se centra en los cuatro tipos mencionados, resulta fundamental comprender su clasificación general, teniendo en cuenta que estas denominaciones se mantienen en inglés por tratarse de términos técnicos estandarizados:

Figura 3. Representación de la Clasificación general de las bases de datos



A. Bases de datos relacionales (relational)

Las bases de datos relacionales constituyen el modelo tradicional más utilizado en el diseño de sistemas de información. Estas se fundamentan en el modelo relacional, en el cual la información se organiza en tablas compuestas por filas y columnas, estableciendo relaciones entre los datos mediante claves primarias y foráneas.

Este tipo de bases de datos se caracteriza por garantizar la integridad, consistencia y estructuración de la información, lo que las hace ideales para sistemas que requieren alta precisión, como sistemas financieros, administrativos y empresariales. Además, utilizan el lenguaje SQL (Structured Query Language) para la manipulación y consulta de datos.

Desde el punto de vista conceptual, las bases de datos relacionales permiten representar entidades del mundo real mediante tablas, donde cada registro corresponde a una instancia de la entidad y cada atributo define sus características.

B. Bases de datos NoSQL

Las bases de datos NoSQL (Not Only SQL) surgen en respuesta a la evolución de las aplicaciones digitales modernas, las cuales requieren gestionar información diversa y cambiante. Este enfoque se aparta de las estructuras tradicionales al permitir una mayor adaptación del almacenamiento de datos según las necesidades específicas de cada sistema.

Estas bases de datos se orientan a entornos dinámicos donde la agilidad en el manejo de la información es prioritaria, facilitando la integración con aplicaciones distribuidas y plataformas tecnológicas actuales. Su diseño favorece la rapidez en el acceso a los datos y la capacidad de respuesta ante grandes volúmenes de información generados de forma continua.

Desde una perspectiva general, NoSQL propone una forma alternativa de organizar y gestionar los datos, enfocándose en la flexibilidad y en la adecuación del modelo de almacenamiento al uso real de la información, más que en estructuras estrictamente predefinidas.

A continuación, se presentan los principales tipos de bases de datos NoSQL y algunos ejemplos representativos:

Tabla 3. Tipos de bases de datos NoSQL

Tipo	Descripción	Ejemplo
Clave-valor	Almacena datos en pares clave-valor.	Redis.
Documentales	Manejan datos en formato JSON o similar.	MongoDB.
Columnar	Organizan datos por columnas.	Cassandra.
Grafos	Representan relaciones complejas.	Neo4j.

C. Bases de datos In-Memory

Estas bases de datos se caracterizan por almacenar la información directamente en la memoria RAM en lugar de hacerlo en disco, lo que permite un acceso extremadamente rápido a los datos. Este tipo de bases de datos es ideal para aplicaciones que requieren procesamiento en tiempo real, como sistemas financieros, análisis de datos en vivo o plataformas de alta concurrencia.

El uso de memoria principal reduce significativamente los tiempos de lectura y escritura, mejorando el rendimiento del sistema. Sin embargo, este enfoque requiere mecanismos adicionales para garantizar la persistencia de los datos, ya que la memoria RAM es volátil.

La siguiente es una comparación de este tipo de bases de datos:

Tabla 4. Comparación de bases de datos In-Memory

Acción	Base de datos en memoria	Base de datos que prioriza la memoria	Base de datos en disco
Rendimiento	Suele ser el más rápido debido al acceso directo a la memoria que reduce la latencia de E/S del disco.	Más rápido que el basado en disco, pero puede no ser tan rápido como el puramente en memoria debido a la posible latencia de E/S del disco.	Normalmente más lento debido a la latencia de E/S del disco.
Coste	Suele ser más costosa debido al elevado coste de la RAM. (La RAM suele ser sólo una parte del coste total).	Coste medio. Se puede aumentar la RAM con almacenamiento en disco más económico.	Suelen ser menos costosas debido a su dependencia del almacenamiento en disco.
Persistencia de datos	A menudo volátiles. Los datos pueden perderse en caso de reinicio o fallo si no se utilizan las funciones de durabilidad.	Proporciona persistencia, lo que reduce el riesgo de pérdida de datos a pesar de depender principalmente de la memoria.	Alta persistencia. Los datos se almacenan, aunque el sistema se apague.
Escalabilidad	Limitado por la RAM disponible a menos que sea	Mayor escalabilidad, ya que puede utilizar	Puede almacenar datos en discos de

Acción	Base de datos en memoria	Base de datos que prioriza la memoria	Base de datos en disco
	posible el escalado horizontal.	almacenamiento en disco para conjuntos de datos más grandes.	gran tamaño, pero es posible que no pueda mantener el ritmo de las demandas de E/S.
Patrones de acceso a los datos	Lo mejor para cargas de trabajo con altas tasas de operación y baja latencia. La mayoría están optimizados para el almacenamiento transitorio de datos.	Adecuado para cargas de trabajo con una combinación de operaciones de lectura y escritura. Requisitos de latencia de bajos a moderados.	La mejor opción para cargas de trabajo analíticas, de almacenamiento a largo plazo o que requieran mucha escritura, o si el rendimiento no es un problema.
Casos prácticos	Análisis en tiempo real, almacenamiento en caché, almacenamiento de sesiones o cualquier cosa transitoria.	Uso general, incluidas aplicaciones en tiempo real y casi real, almacenamiento en caché y cargas de trabajo mixtas.	Almacenamiento de datos a gran escala y aplicaciones con requisitos que no cambian con frecuencia.
Ejemplos	Couchbase Capella, solo memoria. Servidor Couchbase, efímero.	CouchStore o Magma (disponible tanto en Couchbase Capella	Despliegues típicos de SQL Server, Oracle, Postgres, MySQL, etc. (Estos pueden utilizar memoria para

Acción	Base de datos en memoria	Base de datos que prioriza la memoria	Base de datos en disco
		como en Couchbase Server).	almacenar en búfer y caché los planes de consulta y algunos pueden tener complementos para aumentar el almacenamiento en caché). Comparar con NoSQL.

D. Bases de datos NewSQL

Estas bases de datos representan una evolución del modelo relacional, combinando las ventajas de las bases de datos tradicionales (consistencia e integridad) con las capacidades de escalabilidad y alto rendimiento de los sistemas NoSQL.

Este tipo de bases de datos está diseñado para soportar aplicaciones modernas que requieren grandes volúmenes de datos y transacciones en tiempo real, sin sacrificar la estructura y confiabilidad del modelo relacional.

Las bases de datos NewSQL utilizan arquitecturas distribuidas que permiten escalar horizontalmente, manteniendo al mismo tiempo las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad).

Basado en lo anterior, a continuación, se presenta una comparación general de estos tipos de bases de datos abordados, destacando sus características en cuanto a

estructura, escalabilidad, velocidad y principales casos de uso, con el fin de facilitar la comprensión de sus diferencias y aplicaciones más comunes:

Tabla 5. Comparación de bases de datos

Tipo	Estructura	Escalabilidad	Velocidad	Uso principal
Relacional (relational)	Tablas	Media	Media	Sistemas empresariales.
NoSQL	Flexible	Alta	Alta	Big data, apps web.
In-Memory	En RAM	Alta	Muy alta	Tiempo real.
NewSQL	Tablas distribuidas	Alta	Alta	Sistemas modernos.

1.3. Usos, diferencias y ventajas de los distintos tipos de bases de datos

El análisis de los usos, diferencias y ventajas de los distintos tipos de bases de datos constituye un aspecto fundamental en el diseño de sistemas de información, ya que permite seleccionar la tecnología más adecuada según las necesidades del entorno, el volumen de datos, la velocidad requerida y el tipo de aplicación a desarrollar. En la actualidad, la diversidad de modelos de bases de datos responde a la evolución de los sistemas digitales, los cuales demandan soluciones más flexibles, escalables y eficientes.

Cada tipo de base de datos —relacional, NoSQL, in memory y NewSQL— presenta características particulares que influyen directamente en su uso, comportamiento y desempeño. Por ello, comprender sus diferencias no solo facilita su

implementación, sino que también permite optimizar recursos y garantizar soluciones tecnológicas adecuadas.

A. Usos

Los usos de los distintos tipos de bases de datos son:

- **Relacionales (relational)**

Son ampliamente utilizadas en sistemas empresariales tradicionales, como aplicaciones contables, sistemas de gestión administrativa, plataformas bancarias y sistemas de inventario. Su principal fortaleza radica en la estructura organizada de los datos y en la capacidad de mantener relaciones claras entre ellos, lo que permite garantizar la integridad y consistencia de la información. Estas bases de datos son especialmente útiles en entornos donde las transacciones deben ser seguras y confiables.

- **NoSQL**

Han ganado relevancia en aplicaciones modernas, especialmente en sistemas web, redes sociales, plataformas de comercio electrónico y sistemas de análisis de grandes volúmenes de datos (big data). Su uso se orienta a escenarios donde la flexibilidad en la estructura de los datos y la capacidad de escalar horizontalmente son fundamentales. Este tipo de bases de datos permite manejar datos no estructurados o semiestructurados, lo que resulta clave en entornos dinámicos.

- **In-Memory**

Se emplean principalmente en aplicaciones que requieren alta velocidad de procesamiento, como sistemas financieros en tiempo real, análisis de datos en vivo,

videojuegos en línea y sistemas de recomendación. Al trabajar directamente en memoria RAM, reducen significativamente los tiempos de acceso a la información, lo que las hace ideales para entornos donde la latencia debe ser mínima.

- **NewSQL**

Se utilizan en sistemas que requieren combinar la consistencia del modelo relacional con la escalabilidad de los sistemas distribuidos. Son comunes en aplicaciones modernas de alta concurrencia, como plataformas digitales globales, servicios financieros y sistemas que requieren procesamiento de grandes volúmenes de transacciones sin perder integridad.

B. Diferencias

Las diferencias entre estos tipos de bases de datos se evidencian principalmente en su estructura, forma de almacenamiento, escalabilidad y consistencia de los datos. Mientras las bases de datos relacionales utilizan esquemas rígidos basados en tablas, las NoSQL permiten estructuras flexibles que se adaptan a diferentes tipos de información.

Otra diferencia importante radica en el enfoque de consistencia. Las bases de datos relacionales priorizan la integridad de los datos mediante el cumplimiento de las propiedades ACID, lo que garantiza transacciones seguras. En contraste, muchas bases de datos NoSQL adoptan un enfoque basado en la disponibilidad y la tolerancia a fallos, sacrificando en algunos casos la consistencia inmediata.

Las bases de datos In-Memory se diferencian por su ubicación de almacenamiento, ya que operan en memoria principal en lugar de disco, lo que les permite alcanzar altos niveles de rendimiento. Por su parte, las bases de datos NewSQL

integran características de los sistemas relacionales tradicionales con arquitecturas distribuidas, logrando un equilibrio entre consistencia y escalabilidad.

C. Ventajas

Cada tipo de base de datos ofrece ventajas específicas que responden a distintos requerimientos tecnológicos:

- **Relacionales (relational)**

Se destacan por su robustez, confiabilidad y facilidad para mantener la integridad de los datos, lo que las convierte en una opción segura para sistemas críticos. Además, su uso de SQL facilita la gestión y consulta de la información.

- **NoSQL**

Ofrecen una gran ventaja en términos de flexibilidad y escalabilidad, permitiendo manejar grandes volúmenes de datos distribuidos en múltiples servidores. Esta característica es especialmente útil en aplicaciones que requieren adaptarse rápidamente a cambios en la estructura de la información.

- **In-Memory**

Su principal ventaja es la velocidad, ya que permiten acceder a los datos de manera casi inmediata. Esto mejora significativamente el rendimiento de las aplicaciones y reduce los tiempos de respuesta.

- **NewSQL**

Combinan lo mejor de los dos mundos: la consistencia de las bases de datos relacionales y la escalabilidad de los sistemas NoSQL. Esto las convierte en una opción

ideal para aplicaciones modernas que requieren alto rendimiento sin sacrificar la integridad de los datos.

D. Criterios para seleccionar un tipo de base de datos

La elección del tipo de base de datos depende de múltiples factores, entre los que se destacan el volumen de datos, la velocidad de procesamiento requerida, el nivel de consistencia necesario, la estructura de la información y la capacidad de escalabilidad del sistema. En sistemas tradicionales, donde la precisión y la integridad son prioritarias, las bases de datos relacionales suelen ser la mejor opción. En entornos donde se manejan grandes volúmenes de datos y se requiere flexibilidad, las bases de datos NoSQL resultan más adecuadas. Para aplicaciones en tiempo real, las bases de datos In-Memory ofrecen un rendimiento superior, mientras que las bases de datos NewSQL son ideales para sistemas modernos que requieren equilibrio entre rendimiento y consistencia.

2. Fundamentos de bases de datos relacionales

Los fundamentos de las bases de datos relacionales constituyen el eje central para el diseño, implementación y gestión de sistemas de información estructurados. Este modelo, ampliamente adoptado en entornos empresariales y académicos, permite organizar la información de manera lógica mediante estructuras tabulares que garantizan la integridad, consistencia y accesibilidad de los datos.

El modelo relacional se basa en la representación de la información a través de relaciones, comúnmente conocidas como tablas, en las cuales los datos se organizan en filas y columnas. Cada fila representa un registro o instancia de una entidad, mientras que cada columna corresponde a un atributo que describe una característica específica de dicha entidad.

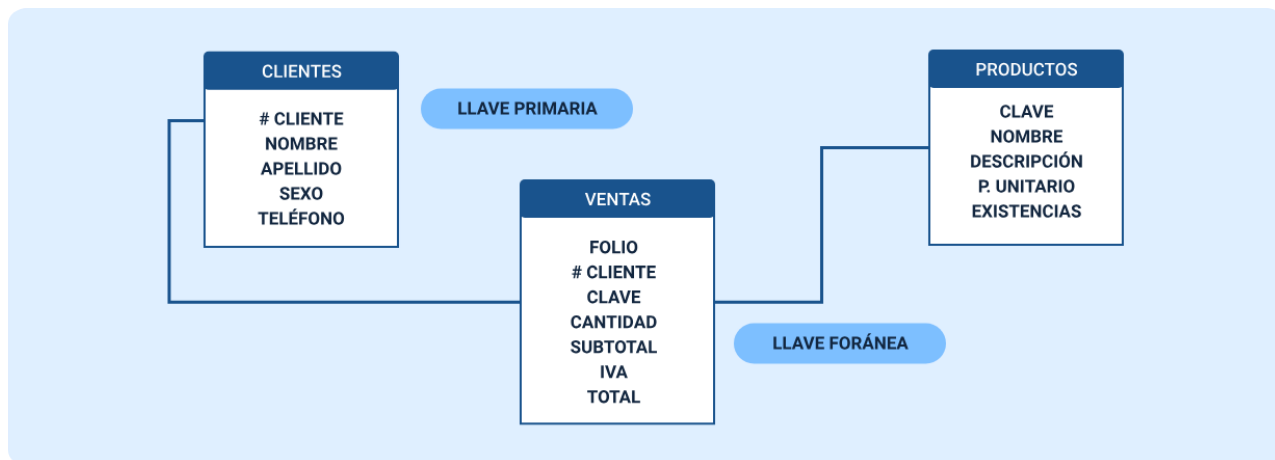
Desde una perspectiva conceptual, este modelo permite abstraer la realidad en estructuras organizadas, facilitando la comprensión del sistema y la manipulación de la información mediante operaciones definidas. Su importancia radica en que proporciona una base sólida para el diseño de bases de datos eficientes, evitando redundancias y asegurando la coherencia de los datos.

Los siguientes son los elementos de una tabla relacional:

- **Tabla:** estructura que almacena datos organizados.
- **Fila (registro):** representa una instancia de la entidad.
- **Columna (atributo):** define una característica del dato.
- **Clave primaria:** identifica de manera única cada registro.
- **Clave foránea:** relaciona una tabla con otra.

Para mayor claridad, se relaciona la representación de esta base de datos con la siguiente figura:

Figura 4. Representación de una estructura básica de una base de datos relacional



Representación de una estructura básica de una base de datos relacional

- CLIENTES
 - # CLIENTE - LLAVE PRIMARIA
 - NOMBRE
 - APELLIDO
 - SEXO
 - TELÉFONO
- VENTAS
 - FOLIO
 - # CLIENTE - LLAVE SECUNDARIA
 - CLAVE
 - CANTIDAD
 - SUBTOTAL

- IVA
- TOTAL
- PRODUCTOS
 - CLAVE
 - NOMBRE
 - DESCRIPCIÓN
 - P. UNITARIO
 - EXISTENCIAS

E. Principios del modelo relacional

El modelo relacional se fundamenta en principios matemáticos derivados de la teoría de conjuntos y la lógica formal, lo que permite definir estructuras precisas y operaciones consistentes sobre los datos. Este enfoque garantiza que la información pueda ser manipulada de manera estructurada, permitiendo realizar consultas, actualizaciones y eliminaciones sin comprometer la integridad del sistema.

Uno de los principios clave es la independencia lógica de los datos, que permite modificar la estructura de la base de datos sin afectar las aplicaciones que la utilizan. Asimismo, el modelo relacional promueve la eliminación de redundancia, lo que reduce inconsistencias y mejora la calidad de la información.

Otro aspecto fundamental es la integridad de los datos, la cual se garantiza mediante restricciones que aseguran que la información almacenada sea válida, coherente y completa. Estas restricciones incluyen claves primarias, claves foráneas y reglas de dominio.

F. Concepto de relación entre datos

En el modelo relacional, las relaciones permiten vincular información entre diferentes tablas. Estas relaciones se establecen mediante claves, lo que permite mantener la coherencia de los datos y evitar duplicidad de información.

Existen diferentes tipos de relaciones, como uno a uno, uno a muchos y muchos a muchos, las cuales determinan cómo se conectan las entidades dentro del sistema. Estas relaciones son fundamentales para estructurar correctamente la base de datos y facilitar la recuperación de información.

2.1. Teoría de bases de datos relacionales

Esta teoría constituye el fundamento conceptual y matemático sobre el cual se construyen los sistemas de gestión de datos modernos. Este modelo fue propuesto por Edgar F. Codd en la década de 1970 y se basa en principios de la teoría de conjuntos y la lógica de predicados, lo que permite estructurar la información de manera formal, consistente y manipulable mediante operaciones bien definidas.

Desde esta perspectiva, una base de datos relacional no se concibe únicamente como un conjunto de tablas, sino como un sistema estructurado de relaciones matemáticas que permiten representar entidades del mundo real y sus interacciones. Esta formalización facilita la manipulación de los datos mediante lenguajes como SQL, garantizando coherencia, integridad y flexibilidad en su gestión.

A. Concepto de relación en el modelo relacional

En la teoría relacional, una relación corresponde a una tabla que contiene un conjunto de tuplas (filas), donde cada tupla representa una instancia única de una entidad. Cada columna de la tabla corresponde a un atributo, el cual define una propiedad específica de dicha entidad.

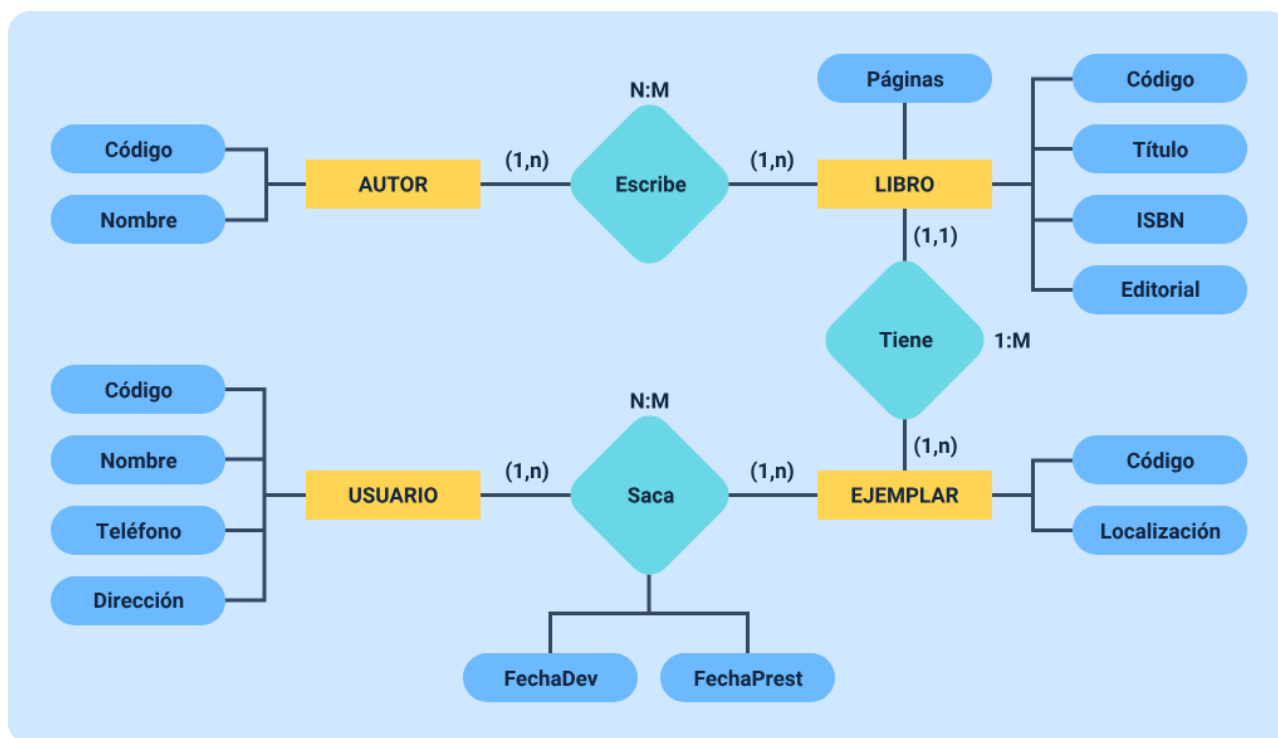
Una característica esencial de las relaciones es que no deben contener duplicados, lo que implica que cada fila debe ser única. Además, el orden de las filas y columnas no afecta el significado de la relación, lo que refuerza el carácter abstracto del modelo.

El modelo relacional se compone de varios elementos clave que permiten estructurar la información de manera lógica y coherente. Entre estos se encuentran:

- **Relación:** conjunto de datos organizados en forma de tabla.
- **Tupla:** fila de la tabla que representa un registro.
- **Atributo:** columna que define una característica del dato.
- **Dominio:** conjunto de valores permitidos para un atributo.
- **Clave:** identificador único de cada tupla.

Asimismo, su representación se puede ejemplificar de la siguiente manera:

Figura 5. Representación conceptual de una relación



Representación conceptual de una relación

AUTOR

Código

Nombre

(1,n)

N:M

Escribe

(1,n)

LIBRO

Páginas

Código

Nombre

Título

ISBN

Editorial

(1,1)

Tiene

1:N

(1.n)

EJEMPLAR

Código

Localización

(1:n)

Saca

N:M

FechaDev

FechaPrest

(1:n)

USUARIO

Código

Nombre

Teléfono

Dirección

B. Fundamentos de la teoría relacional

La teoría relacional establece los principios que permiten organizar y garantizar la calidad de los datos dentro de una base de datos. Entre estos principios se encuentran los conceptos de dominio y restricciones, que definen los valores válidos para cada atributo y evitan el ingreso de información incorrecta. Asimismo, las claves cumplen un papel central al permitir la identificación única de los registros y la correcta relación entre tablas, asegurando la integridad de entidad y la integridad referencial.

Estos mecanismos trabajan de manera conjunta para preservar la consistencia de la información, reducir errores y evitar redundancias. Además, la teoría relacional proporciona las bases conceptuales para la normalización de datos, proceso orientado a estructurar la información de forma lógica y eficiente, facilitando su mantenimiento y actualización en sistemas complejos.

C. Manipulación y aplicación de los datos relacionales

La manipulación de la información en bases de datos relacionales se sustenta en el álgebra y el cálculo relacional, los cuales permiten definir consultas y operaciones sobre los datos. Mientras el álgebra relacional describe cómo obtener los resultados

mediante operaciones formales, el cálculo relacional se centra en especificar qué información se desea recuperar, enfoque que da origen a lenguajes de consulta como SQL.

En la práctica, estos fundamentos permiten desarrollar sistemas confiables y escalables, capaces de gestionar grandes volúmenes de datos en entornos empresariales, educativos y financieros. La aplicación adecuada de la teoría relacional contribuye a la creación de soluciones tecnológicas robustas, con información coherente, segura y fácil de administrar.

2.2. Componentes de una base de datos

El adecuado funcionamiento de una base de datos relacional no depende únicamente de su estructura, sino también de la forma en que se gestionan aspectos clave como la redundancia, la inconsistencia y la integridad de los datos. Estos componentes representan factores críticos en la calidad de la información, ya que influyen directamente en la confiabilidad, eficiencia y mantenimiento del sistema.

En el diseño de bases de datos, estos elementos deben ser cuidadosamente analizados y controlados, debido a que una mala gestión puede generar errores, duplicidad de información y dificultades en la toma de decisiones. Por ello, comprender su naturaleza, causas y efectos permite diseñar sistemas más robustos y eficientes.

A continuación, se explica en qué consiste cada uno de estos componentes:

- **Redundancia de datos**

Se refiere a la duplicación innecesaria de información dentro de una base de datos. Este fenómeno ocurre cuando un mismo dato se almacena en múltiples lugares sin una justificación válida, lo que puede generar problemas de almacenamiento, mantenimiento y actualización.

Desde una perspectiva técnica, la redundancia puede surgir por un diseño inadecuado de la base de datos, donde no se han aplicado correctamente principios como la normalización. Aunque en algunos casos la redundancia puede ser intencional para mejorar el rendimiento, en la mayoría de los escenarios representa una debilidad en la estructura del sistema.

Por ejemplo, si en varias tablas se almacena repetidamente la información de un cliente (nombre, dirección, teléfono), se genera redundancia que puede afectar la eficiencia del sistema.

Tabla 6. Redundancia de datos

ID Cliente	Nombre	Ciudad
1	Juan	Ibagué
2	Ana	Ibagué
3	Pedro	Ibagué

En este caso, el valor "Ibagué" se repite innecesariamente, lo que podría optimizarse mediante una estructura relacional más adecuada.

- **Inconsistencia de datos**

Se presenta cuando existen discrepancias en la información almacenada; es decir, cuando un mismo dato presenta diferentes valores en distintas partes de la base de datos. Este problema está directamente relacionado con la redundancia, ya que la duplicación de datos aumenta la probabilidad de inconsistencias.

Cuando los datos no son coherentes, se pierde la confiabilidad del sistema, afectando procesos como consultas, reportes y toma de decisiones. Por ejemplo, si un cliente tiene dos direcciones diferentes registradas en distintas tablas, el sistema no podrá determinar cuál es la correcta.

La inconsistencia suele originarse por actualizaciones parciales, errores humanos o falta de mecanismos de control en la base de datos.

- **Integridad de los datos**

Hace referencia a la precisión, coherencia y validez de la información almacenada en una base de datos. Es uno de los pilares fundamentales del modelo relacional, ya que garantiza que los datos sean correctos y confiables en todo momento.

Para asegurar la integridad, se establecen diferentes tipos de restricciones que regulan el comportamiento de los datos dentro del sistema. Estas restricciones permiten evitar errores, mantener relaciones válidas entre tablas y asegurar que la información cumpla con reglas definidas.

Aunque existen diferentes enfoques, los principales tipos de integridad en bases de datos relacionales son los siguientes:

- **Integridad de entidad:** garantiza que cada registro tenga una clave primaria única y no nula
- **Integridad referencial:** asegura que las relaciones entre tablas sean válidas
- **Integridad de dominio:** define valores permitidos para cada atributo

A. Relación entre redundancia, inconsistencia e integridad

Estos tres componentes están estrechamente relacionados dentro del diseño de bases de datos. La redundancia excesiva puede generar inconsistencias, y estas a su vez afectan la integridad de los datos. Por esta razón, es fundamental aplicar buenas prácticas de diseño, como la normalización, para reducir la duplicidad de información y garantizar la coherencia del sistema. Un sistema bien diseñado busca minimizar la redundancia, controlar la inconsistencia y asegurar la integridad, logrando así una base de datos eficiente y confiable.

El control de la redundancia, la inconsistencia y la integridad se logra mediante el uso de técnicas y herramientas propias del diseño de bases de datos. Entre ellas se destacan la normalización, el uso de claves primarias y foráneas, la definición de restricciones y la implementación de reglas de negocio. Asimismo, los sistemas gestores de bases de datos (DBMS) incorporan mecanismos automáticos que ayudan a mantener la integridad de los datos, evitando operaciones que puedan comprometer la coherencia del sistema.

En sistemas reales, estos componentes son fundamentales para garantizar la calidad de la información. En sectores como el financiero, la salud o la educación, una inconsistencia o error en los datos puede generar consecuencias graves, lo que hace indispensable contar con bases de datos bien estructuradas. El control de estos

aspectos permite desarrollar sistemas confiables, reducir errores y mejorar la toma de decisiones basada en datos.

2.3. Sistema gestor de bases de datos

El sistema gestor de bases de datos, conocido como DBMS (Database Management System), es un conjunto de programas que permite crear, administrar, manipular y controlar el acceso a una base de datos. Este sistema actúa como intermediario entre los usuarios o aplicaciones y los datos almacenados, garantizando que la información sea gestionada de manera eficiente, segura y organizada.

En el contexto del diseño y modelado de bases de datos, el DBMS desempeña un papel fundamental, ya que proporciona las herramientas necesarias para implementar las estructuras diseñadas, ejecutar consultas, asegurar la integridad de los datos y controlar el acceso a la información.

A. Funcionamiento de un DBMS

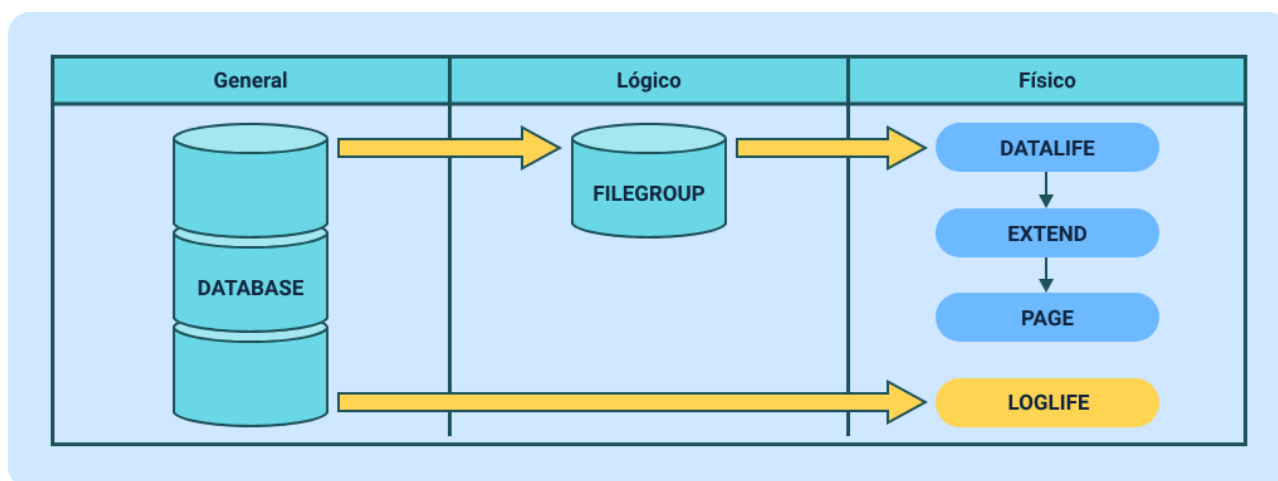
Un sistema gestor de bases de datos cumple diversas funciones que permiten el manejo integral de la información:

- **Definición de datos:** permite crear estructuras como tablas, índices y relaciones mediante lenguajes como SQL.
- **Manipulación de datos:** facilita la inserción, actualización, eliminación y consulta de información.
- **Control de acceso:** gestiona permisos y roles de usuarios para proteger la información.

- **Gestión de transacciones:** asegura que las operaciones se realicen de manera segura y consistente.
- **Recuperación ante fallos:** permite restaurar la base de datos en caso de errores o fallos del sistema.
- **Optimización de consultas:** mejora el rendimiento en la recuperación de información.

La siguiente figura explica cómo está estructurada la arquitectura de un DBMS

Figura 6. Arquitectura de un DBMS



Un sistema gestor de bases de datos (DBMS) está compuesto por diversos elementos que permiten su funcionamiento, entre los que se encuentran el motor de base de datos, encargado del almacenamiento y la recuperación de la información; el lenguaje de consulta, como SQL, que permite interactuar con los datos; el administrador de transacciones, responsable de garantizar la consistencia de las operaciones; el gestor de almacenamiento, que define cómo se guardan los datos en disco o memoria; y el diccionario de datos, que contiene la definición de la estructura de la base de datos. Según el modelo de datos que utilizan, los DBMS pueden

clasificarse en relacionales, orientados a tablas y relaciones; NoSQL, basados en estructuras flexibles; distribuidos, que gestionan datos en múltiples ubicaciones; y en memoria, que almacenan la información en RAM para ofrecer mayor velocidad de acceso.

Igualmente, se destacan algunos procesos positivos y negativos como son:

- **Ventajas**

- Permite la centralización de datos, evitando duplicidad innecesaria.
- Mejora la seguridad, controlando accesos y permisos.
- Garantiza la integridad y consistencia de la información.
- Facilita la recuperación de datos ante fallos.
- Optimiza el rendimiento en consultas.
- Permite la concurrencia, es decir, múltiples usuarios accediendo simultáneamente.

- **Desventajas**

- Requieren recursos computacionales elevados.
- Su implementación puede ser compleja.
- Necesitan personal capacitado para su administración.
- Pueden generar costos de licencia en algunos casos.

Es importante conocer los diferentes lenguajes que utiliza DBMS para interactuar con la base de datos:

- **DDL (Data Definition Language)**

Permite definir y modificar la estructura de la base de datos, incluyendo la creación, alteración y eliminación de tablas, vistas e índices.

- **DML (Data Manipulation Language)**

Se utiliza para gestionar los datos almacenados, permitiendo insertar nuevos registros, actualizar información existente y eliminar datos cuando sea necesario.

- **DCL (Data Control Language)**

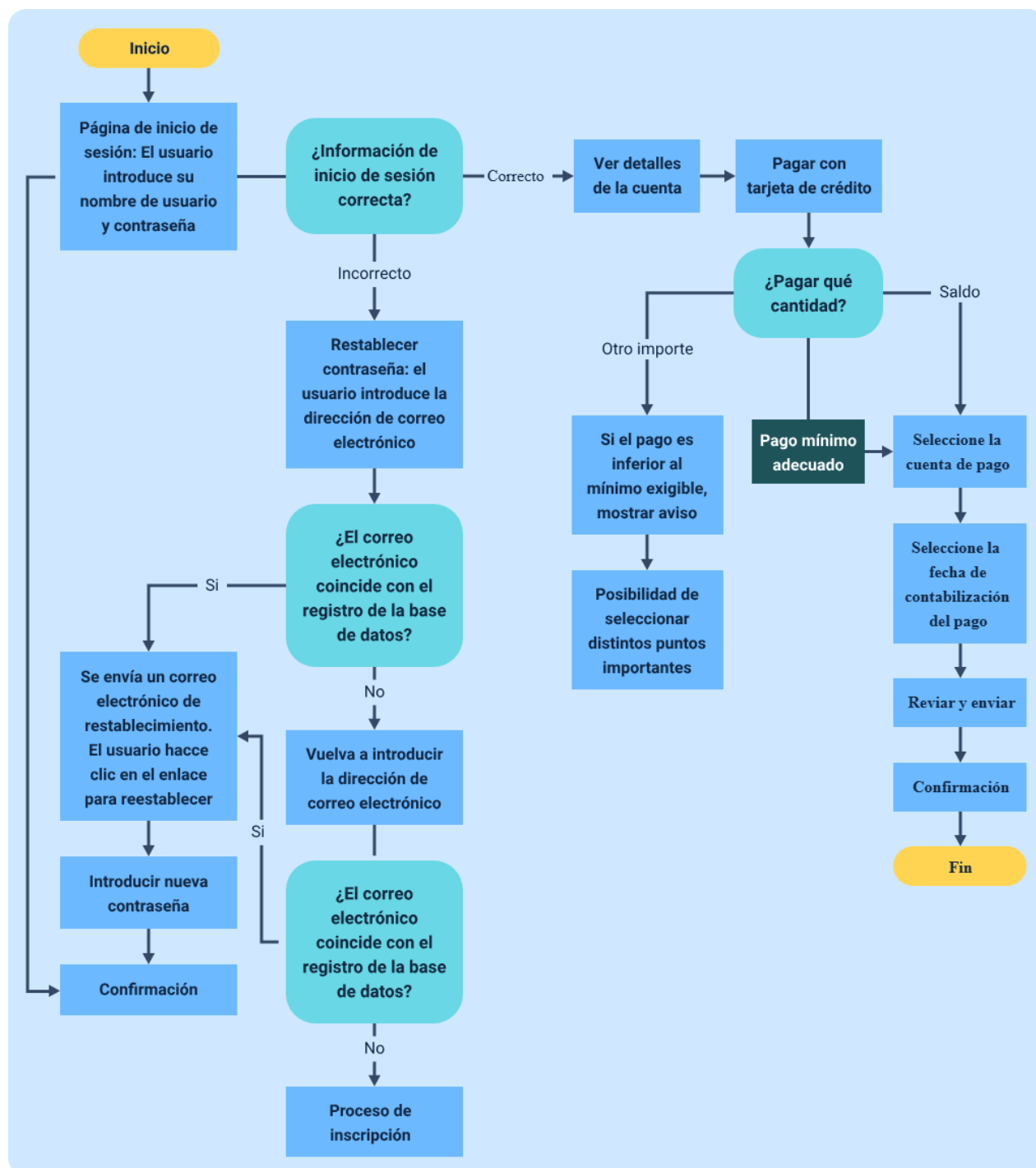
Controla el acceso a la base de datos mediante la asignación y revocación de permisos, garantizando la seguridad y el uso adecuado de la información.

- **TCL (Transaction Control Language)**

Gestiona las transacciones dentro de la base de datos, permitiendo confirmar, deshacer o agrupar operaciones para asegurar la consistencia e integridad de los datos.

Para complementar lo anterior y tener mayor claridad sobre los DBMS, se presenta el siguiente esquema:

Figura 7. Interacción usuario – DBMS – base de datos



2.4. Modelos de diseño de bases de datos

El diseño de bases de datos es un proceso fundamental dentro del desarrollo de sistemas de información, ya que permite estructurar, organizar y representar los datos de manera eficiente antes de su implementación. Para lograrlo, se emplean distintos modelos de diseño que permiten abstraer la realidad en diferentes niveles de detalle, facilitando la comprensión, análisis y construcción de la base de datos.

Estos modelos se clasifican en tres niveles principales: conceptual, lógico y físico. Cada uno cumple una función específica dentro del proceso de diseño, permitiendo pasar de una visión general del problema a una implementación técnica concreta.

El uso de modelos de diseño permite:

- Comprender la estructura de los datos antes de implementarlos.
- Identificar relaciones entre entidades.
- Evitar errores en el diseño de la base de datos.
- Optimizar el almacenamiento y acceso a la información.
- Facilitar la comunicación entre analistas, desarrolladores y usuarios.

A. Modelo conceptual

Representa el nivel más abstracto del diseño de bases de datos. En esta etapa, se identifican las entidades, sus atributos y las relaciones entre ellas, sin considerar aspectos técnicos de implementación. Su propósito es describir la realidad del problema de manera clara y comprensible para todos los actores involucrados.

Este modelo se enfoca en responder qué información se necesita y cómo se relaciona, sin preocuparse por cómo será almacenada. Es utilizado principalmente en la fase de análisis de requerimientos.

Los elementos del modelo conceptual son:

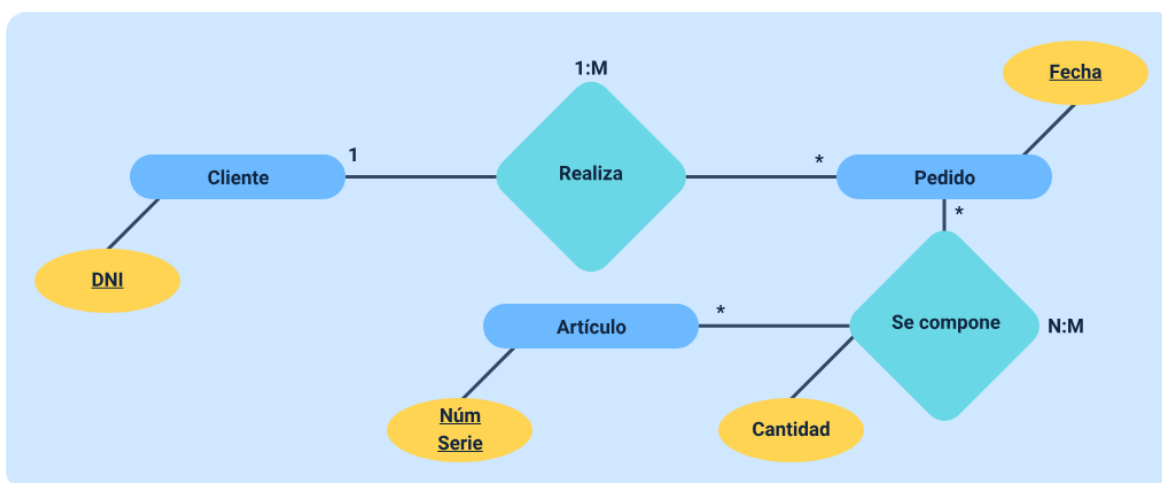
- **Entidades:** representan objetos o elementos del mundo real (cliente, producto).
- **Atributos:** características de las entidades (nombre, precio).
- **Relaciones:** asociaciones entre entidades (un cliente compra un producto).

Como características tiene:

- Es independiente de cualquier DBMS
- Se enfoca en la comprensión del negocio
- No incluye detalles técnicos
- Facilita la comunicación con usuarios no técnicos

Este modelo puede ejemplificarse de la siguiente manera:

Figura 8. Ejemplo modelo entidad-relación



Ejemplo modelo entidad-relación

Cliente

DNI

1

Realiza

1:N

*Pedido

Fecha

Se compone

N:M

Cantidad

*Artículo

Núm

Serie

B. Modelo lógico

El modelo lógico representa un nivel intermedio entre el modelo conceptual y el físico. En esta etapa, la información se estructura en tablas, definiendo atributos, tipos de datos y relaciones, pero aún sin considerar detalles específicos del sistema gestor de bases de datos.

Aquí se traduce el modelo conceptual a una estructura más formal, basada en el modelo relacional, donde se definen claves primarias, claves foráneas y restricciones.

Los elementos del modelo lógico son:

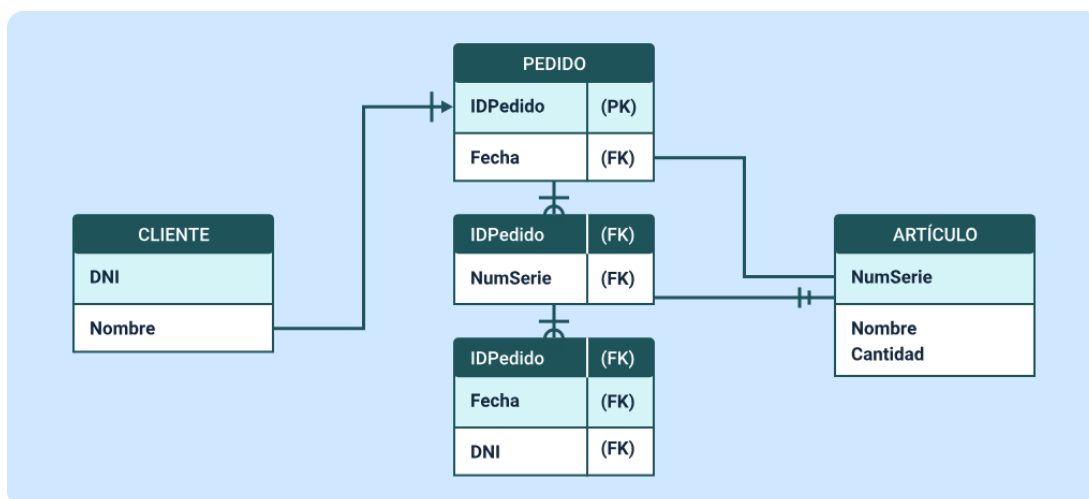
- **Tablas:** representan entidades.
- **Campos:** atributos de las tablas.
- **Claves primarias:** identifican registros.
- **Claves foráneas:** establecen relaciones.
- **Restricciones:** garantizan integridad.

Sus características son:

- Representa la estructura de datos de forma organizada.
- Es independiente del DBMS específico.
- Permite validar relaciones entre datos.
- Define reglas de integridad.

Este modelo igualmente se puede ejemplificar basado en lo siguiente:

Figura 9. Ejemplo modelo lógico



Ejemplo modelo lógico

CLIENTE

- DNI (PK)
- Nombre

PEDIDO

- IDPedido (PK)
- Fecha (FK)

Tabla intermedia (relación Pedido—Artículo)

- IDPedido (FK)
- NumSerie (FK)

ARTÍCULO

- NumSerie (PK)
- Nombre
- Cantidad

Relaciones representadas

- CLIENTE — PEDIDO (1:N)
- PEDIDO — ARTÍCULO (N:M, mediante tabla intermedia)

C. Modelo físico

Corresponde al nivel más detallado del diseño, donde se define cómo se implementará la base de datos en un sistema gestor específico. En esta etapa se consideran aspectos como almacenamiento, índices, rendimiento y optimización.

Este modelo traduce el diseño lógico a estructuras reales dentro del DBMS, incluyendo tipos de datos específicos, creación de tablas y definición de índices.

Los elementos del modelo físico son:

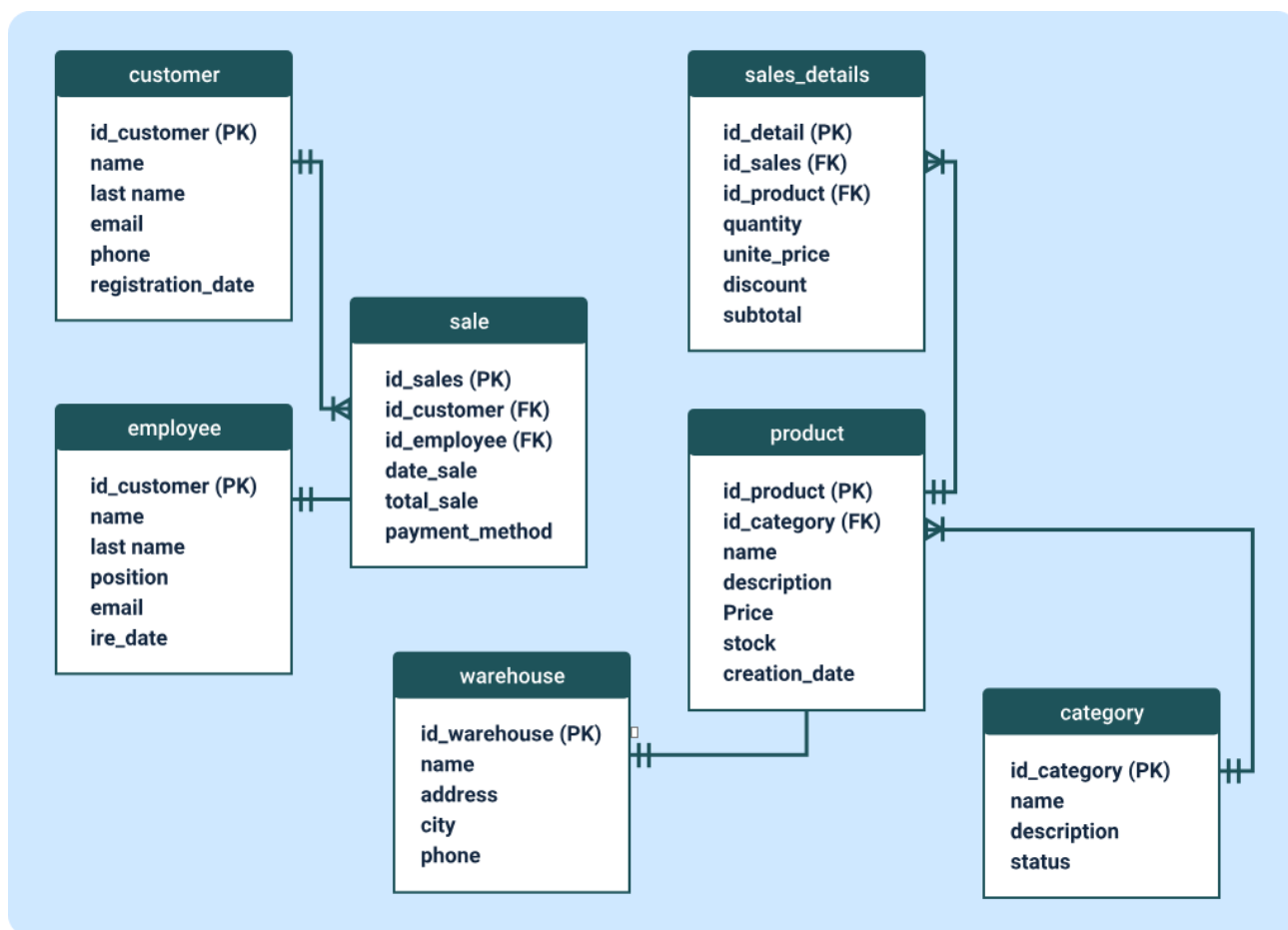
- **Tablas físicas:** estructuras reales creadas en el DBMS donde se almacenan los datos.
- **Tipos de datos específicos:** definen el formato y tamaño de cada campo (INT, VARCHAR, DATE, etc.).
- **Índices:** mecanismos que mejoran la velocidad de acceso y consulta de la información.
- **Restricciones de integridad:** reglas que garantizan la validez y consistencia de los datos.

Sus características destacan lo siguiente:

- Depende del DBMS utilizado.
- Considera el rendimiento y la optimización del sistema.
- Define cómo se almacenan físicamente los datos.
- Permite una implementación eficiente y segura de la base de datos.

Su ejemplificación puede darse como lo que se relaciona en la siguiente imagen:

Figura 10. Ejemplo modelo físico (SQL)



Ejemplo de implementación física

```

CREATE TABLE Cliente (

    ID_Cliente INT PRIMARY KEY,

    Nombre VARCHAR(50),

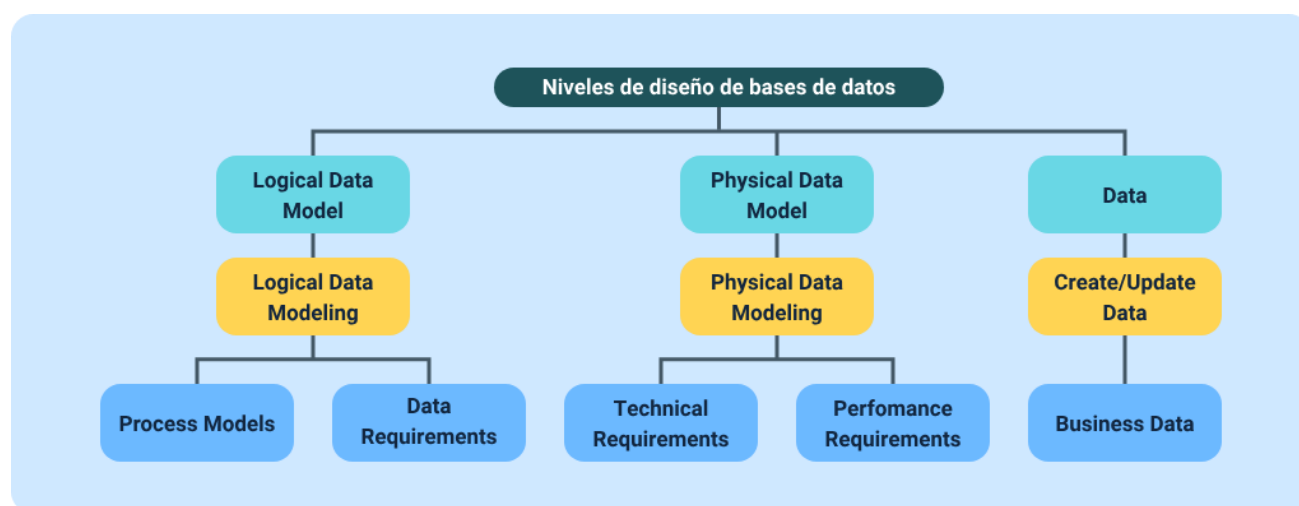
    Ciudad VARCHAR(50)

);
    
```

Este ejemplo muestra cómo el modelo lógico se transforma en una estructura implementada en un DBMS.

Para finalizar, se detallan los niveles del diseño de bases de datos. Los modelos y conceptos que se incluyen en el esquema, se mantienen en idioma inglés, ya que corresponden a la denominación original utilizada en la literatura técnica y en los estándares internacionales del área:

Figura 11. Niveles de diseño de bases de datos



3. Diseño de bases de datos relacionales

Es un proceso fundamental en el desarrollo de sistemas de información, ya que permite estructurar los datos de manera organizada, coherente y eficiente, garantizando su integridad, consistencia y facilidad de acceso. Este proceso se basa en la transformación de los requerimientos del negocio en un modelo estructurado que permita almacenar y gestionar la información de forma óptima.

A diferencia de otras etapas del desarrollo, el diseño de bases de datos no solo implica definir tablas, sino también establecer relaciones entre los datos, identificar restricciones y asegurar que la información sea manejada de manera adecuada dentro del sistema. Un diseño correcto permite evitar problemas como redundancia, inconsistencias y bajo rendimiento en las consultas.

El proceso de diseño relacional inicia con la comprensión del problema y la identificación de las entidades principales que intervienen en el sistema. Posteriormente, estas entidades se transforman en tablas, definiendo sus atributos y estableciendo relaciones mediante claves primarias y foráneas. Este enfoque permite representar de manera estructurada la realidad del sistema.

Uno de los aspectos más importantes en el diseño de bases de datos relacionales es la organización de la información mediante el modelo relacional, el cual permite estructurar los datos en tablas interconectadas. Esta estructura facilita la manipulación de la información mediante lenguajes de consulta como SQL, permitiendo realizar operaciones de inserción, actualización, eliminación y consulta de datos.

Asimismo, el diseño relacional debe garantizar la integridad de los datos, lo que implica que la información almacenada sea válida, coherente y consistente. Para ello, se

establecen restricciones que controlan el comportamiento de los datos, evitando errores y asegurando la calidad de la información.

Otro elemento clave dentro de este proceso es la normalización, la cual permite organizar los datos de manera que se reduzca la redundancia y se eviten anomalías en la manipulación de la información. A través de este proceso, las tablas se estructuran en diferentes formas normales que optimizan el almacenamiento y mejoran la eficiencia del sistema.

3.1. Normalización de bases de datos

La normalización de bases de datos es un proceso fundamental dentro del diseño relacional, cuyo objetivo principal es organizar la información de manera eficiente, eliminando redundancias innecesarias y evitando problemas de inconsistencia en los datos. Este proceso permite estructurar las tablas de tal forma que cada dato se almacene una sola vez, garantizando integridad, coherencia y facilidad de mantenimiento en el sistema.

Desde una perspectiva teórica, la normalización se basa en un conjunto de reglas conocidas como formas normales, las cuales establecen criterios específicos para la organización de los datos. Estas reglas permiten transformar una base de datos inicial, generalmente desorganizada o redundante, en una estructura optimizada que facilita su gestión.

La normalización es clave para garantizar la calidad de la información y evitar anomalías en las operaciones de inserción, actualización y eliminación.

A. Importancia de la normalización

En el diseño de bases de datos relacionales, la normalización permite mejorar significativamente la estructura del sistema. Un diseño no normalizado puede generar duplicidad de datos, inconsistencias y dificultades en la actualización de la información.

Entre sus principales beneficios se encuentra la capacidad de:

- Evitar la duplicación de datos.
- Mantener la coherencia de la información.
- Facilitar la actualización de registros.
- Mejorar la organización de las tablas.
- Reducir errores en el sistema.

Hay que tener presente que cuando una base de datos no está correctamente estructurada, pueden presentarse anomalías que afectan su funcionamiento. Estas anomalías se clasifican en tres tipos principales:

- **Anomalía de inserción**

Ocurre cuando no es posible agregar nuevos datos a la base de datos sin incluir información adicional que no es necesaria en ese momento. Esto limita el registro de información independiente, obliga a completar campos irrelevantes y puede generar datos incompletos o inconsistentes.

- **Anomalía de actualización**

Se presenta cuando un mismo dato debe modificarse en varios registros o tablas al mismo tiempo. Si la actualización no se realiza de manera uniforme, se producen inconsistencias que afectan la confiabilidad y exactitud de la información almacenada.

- **Anomalía de eliminación**

Sucede cuando al eliminar un registro se pierde información relevante que está asociada a ese dato y que aún podría ser necesaria. Esto provoca pérdida de datos importantes y reduce la integridad de la base de datos.

B. Formas normales

La normalización se desarrolla a través de diferentes niveles llamados formas normales. Cada forma normal establece condiciones que deben cumplirse para mejorar la estructura de la base de datos:

- **Primera Forma Normal (1FN)**

La primera forma normal establece que todos los atributos de una tabla deben contener valores atómicos, es decir, que no se permiten valores repetidos ni grupos de datos dentro de un mismo campo.

- **Segunda Forma Normal (2FN)**

La segunda forma normal se alcanza cuando la tabla cumple con la 1FN y, además, todos los atributos dependen completamente de la clave primaria.

- **Tercera Forma Normal (3FN)**

La tercera forma normal se cumple cuando la tabla está en 2FN y no existen dependencias transitivas, es decir, cuando los atributos no dependen de otros atributos que no sean la clave primaria.

El proceso de normalización se realiza de manera progresiva, partiendo de una estructura inicial y aplicando las reglas de cada forma normal. Este proceso implica:

- Identificar entidades y atributos.
- Definir claves primarias.
- Analizar dependencias funcionales.
- Dividir tablas cuando sea necesario.

3.2. Modelo relacional

El modelo relacional constituye uno de los pilares fundamentales en el diseño de bases de datos, ya que proporciona una estructura lógica que permite organizar la información en forma de tablas relacionadas entre sí. Este modelo, propuesto por Edgar F. Codd, se basa en principios matemáticos derivados de la teoría de conjuntos y la lógica formal, lo que garantiza un manejo consistente, preciso y estructurado de los datos.

En el contexto del diseño de bases de datos relacionales, el modelo relacional permite representar entidades del mundo real mediante relaciones (tablas), facilitando la manipulación de la información a través de operaciones bien definidas. Este enfoque ha sido ampliamente adoptado debido a su simplicidad conceptual, su robustez y su capacidad para mantener la integridad de los datos.

A. Estructura del modelo relacional

El modelo relacional se basa en la organización de los datos en tablas, también denominadas relaciones. Cada tabla está compuesta por filas y columnas, donde las filas representan registros y las columnas representan atributos.

En este modelo, cada tabla posee una estructura definida que permite identificar de manera única cada registro y establecer relaciones con otras tablas. Esta estructura facilita la organización de la información y permite realizar consultas eficientes.

El modelo relacional está estrechamente relacionado con la normalización, ya que ambos buscan optimizar la estructura de la base de datos. Mientras el modelo relacional define la organización de los datos, la normalización se encarga de eliminar redundancias y mejorar la integridad.

La combinación del modelo relacional y la normalización permite diseñar bases de datos eficientes y libres de inconsistencias.

3.3. Tipos de claves

En el modelo relacional, las claves representan uno de los elementos más importantes dentro del diseño de bases de datos, ya que permiten identificar registros de manera única y establecer relaciones entre tablas. Su correcta definición es fundamental para garantizar la integridad, coherencia y organización de la información dentro del sistema.

Las claves no solo cumplen una función de identificación, sino que también permiten estructurar las relaciones entre los datos, evitando duplicidades y asegurando que la información se mantenga consistente en todo momento. En este contexto, los tipos de claves más relevantes son la clave primaria, la clave foránea y las claves candidatas.

Las claves son esenciales para garantizar la unicidad de los datos y la correcta relación entre las tablas dentro de una base de datos.

Basado en lo anterior, los tipos de claves son:

- **Clave primaria**

Es el atributo, o conjunto de atributos, que permite identificar de manera única cada registro dentro de una tabla. Esto significa que no puede existir más de un registro con el mismo valor en la clave primaria, ni tampoco se permiten valores nulos.

Desde el punto de vista del diseño, la clave primaria es el elemento central de cada tabla, ya que define la identidad de cada registro y permite diferenciarlo de los demás. Su correcta selección es fundamental para garantizar la integridad de los datos.

- **Clave foránea**

Es un atributo que permite establecer una relación entre dos tablas. Este tipo de clave hace referencia a la clave primaria de otra tabla, lo que permite vincular información y mantener la coherencia entre los datos.

La clave foránea es esencial para garantizar la integridad referencial, ya que asegura que los valores registrados en una tabla correspondan a datos existentes en otra. Esto evita la creación de relaciones inválidas o inconsistentes dentro de la base de datos.

- **Claves candidatas**

Son aquellos atributos, o conjuntos de atributos, que pueden identificar de manera única cada registro dentro de una tabla. En otras palabras, son posibles opciones para convertirse en clave primaria.

De todas las claves candidatas, se selecciona una como clave primaria, mientras que las demás permanecen como alternativas que podrían cumplir la misma función.

A. Importancia de las claves en el diseño de bases de datos

Las claves desempeñan un papel fundamental en la organización y funcionamiento de una base de datos relacional. Su uso permite estructurar la información de manera lógica, evitando duplicidad de datos y garantizando la integridad del sistema.

Además, las claves facilitan la realización de consultas complejas, ya que permiten relacionar información de diferentes tablas de manera eficiente.

En el diseño de bases de datos, es importante considerar ciertos criterios al definir las claves:

- Seleccionar atributos que sean únicos y estables.
- Evitar el uso de datos que puedan cambiar con el tiempo.
- Definir correctamente las relaciones entre tablas.
- Garantizar que las claves foráneas correspondan a valores existentes.

3.4. Entidades fuertes y débiles

En el diseño de bases de datos relacionales, las entidades constituyen uno de los elementos fundamentales para representar la realidad dentro de un sistema de información. Estas permiten modelar objetos, conceptos o elementos del mundo real que poseen características propias y que pueden ser almacenados en una base de datos. Dentro de esta clasificación, se distinguen dos tipos principales y cuya correcta identificación es esencial para garantizar una estructura coherente y eficiente del modelo de datos:

- **Entidad fuerte**

Es aquella que posee existencia propia dentro del sistema, es decir, puede ser identificada de manera única sin depender de otra entidad. Este tipo de entidad cuenta con una clave primaria que permite distinguir cada uno de sus registros.

Desde el punto de vista del diseño, las entidades fuertes representan los elementos principales del sistema, como clientes, productos, empleados o usuarios. Estas entidades contienen información relevante que puede ser gestionada de manera independiente.

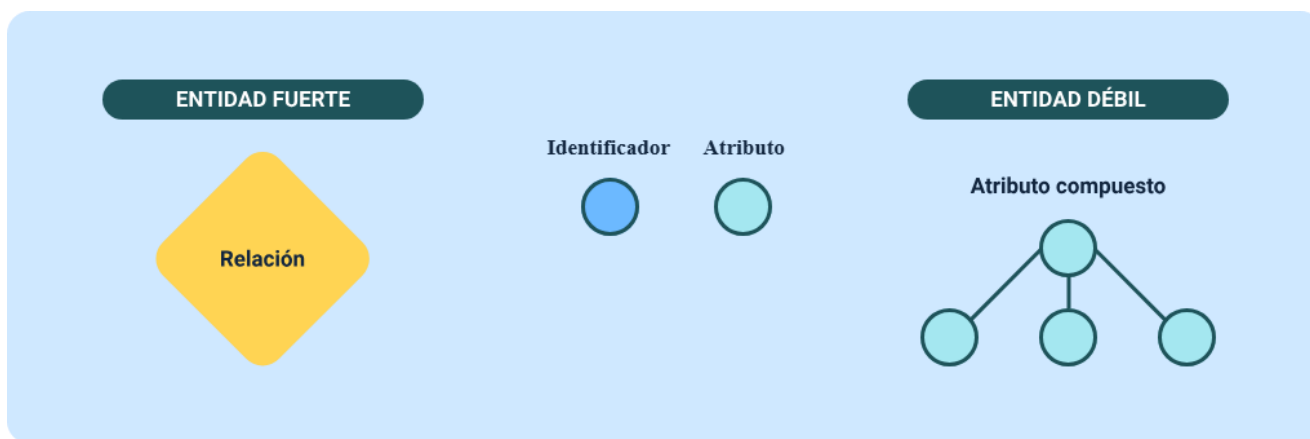
- **Entidad débil**

Es aquella que no puede identificarse de manera única por sí sola, ya que depende de otra entidad (generalmente una entidad fuerte) para su existencia. Este tipo de entidad no posee una clave primaria completa propia, sino que utiliza una clave parcial que, combinada con la clave primaria de la entidad fuerte, permite identificar sus registros.

Las entidades débiles suelen representar elementos que dependen directamente de otros, como detalles de pedidos, registros asociados o elementos subordinados dentro del sistema.

Las anteriores entidades pueden verse representadas de la siguiente manera:

Figura 12. Relación entre entidad fuerte y entidad débil



Relación entre entidad fuerte y entidad débil

ENTIDAD FUERTE

Identificador

Atributo

ENTIDAD DÉBIL

Atributo compuesto

Relación

Las diferencias entre estos tipos de entidades radican principalmente en su independencia y forma de identificación, tal como se presenta en la siguiente tabla:

Tabla 7. Comparación de entidades

Característica	Entidad fuerte	Entidad débil
Existencia	Independiente	Dependiente
Identificación	Clave primaria propia	Clave parcial + clave foránea

Característica	Entidad fuerte	Entidad débil
Relación	No depende de otras	Depende de una entidad fuerte

- **Relación de dependencia**

La relación entre una entidad fuerte y una entidad débil se conoce como relación de dependencia o relación identificadora. En este tipo de relación, la entidad débil no puede existir sin la entidad fuerte, ya que su identificación depende directamente de ella.

Esta relación se representa en los diagramas entidad-relación mediante símbolos específicos, como líneas dobles o rectángulos dobles para indicar la naturaleza dependiente de la entidad débil.

La identificación correcta de entidades fuertes y débiles es fundamental para construir modelos de datos adecuados, ya que permite representar correctamente la estructura del sistema y evitar errores en la implementación.

Además, este concepto facilita la normalización de la base de datos, ya que ayuda a definir correctamente las relaciones entre tablas y evita redundancias innecesarias.

Las entidades fuertes y débiles están directamente relacionadas con:

- Las claves primarias y foráneas.
- La integridad referencial.
- El modelo entidad-relación.
- La normalización de bases de datos.

3.5. Campos relacionales y dependencia de existencia

En el diseño de bases de datos relacionales, los campos relacionales y la dependencia de existencia constituyen elementos fundamentales para establecer la estructura y las relaciones entre las tablas. Estos conceptos permiten definir cómo se vinculan los datos entre sí y garantizan que la información mantenga coherencia, integridad y consistencia dentro del sistema.

Los campos relacionales corresponden a aquellos atributos que permiten establecer relaciones entre tablas, mientras que la dependencia de existencia define el grado en que una entidad o registro depende de otro para existir dentro de la base de datos. Ambos conceptos están estrechamente relacionados con la integridad referencial y con la correcta estructuración del modelo relacional.

Para comprender la organización y coherencia de la información en una base de datos, es fundamental analizar el papel que cumplen los campos relacionales y su impacto en las relaciones, la dependencia de existencia y la integridad referencial:

- **Campos relacionales**

Son atributos dentro de una tabla que establecen una conexión con otra tabla. Generalmente, estos campos corresponden a claves foráneas, las cuales hacen referencia a la clave primaria de otra tabla.

Desde el punto de vista del diseño, estos campos son esenciales para evitar la duplicación de datos y permitir la organización estructurada de la información. En lugar de repetir datos en múltiples tablas, se utiliza un campo relacional que referencia la información en otra tabla, lo que mejora la eficiencia y reduce la redundancia.

- **Relación entre campos relacionales**

Permiten establecer distintos tipos de relaciones, dependiendo de la estructura de los datos. Estas relaciones determinan cómo interactúan las tablas entre sí y cómo se puede acceder a la información relacionada.

Las relaciones pueden ser de uno a uno, uno a muchos o muchos a muchos y su correcta implementación depende del uso adecuado de los campos relacionales.

- **Dependencia de existencia**

Se refiere a la relación en la cual un registro o entidad no puede existir sin la presencia de otro. Este concepto es especialmente relevante en el caso de entidades débiles, donde la existencia de un registro depende directamente de otro registro en una entidad diferente.

Desde una perspectiva estructural, la dependencia de existencia implica que un registro en una tabla solo puede ser creado si existe previamente un registro relacionado en otra tabla. Esto garantiza la coherencia de los datos y evita la creación de registros huérfanos.

- **Relación con la integridad referencial**

Están directamente relacionados con la integridad referencial, la cual asegura que las relaciones entre tablas sean válidas.

Cuando se define una clave foránea, el sistema gestor de bases de datos garantiza que el valor ingresado corresponda a un registro existente en la tabla relacionada. Esto evita inconsistencias y asegura que los datos mantengan coherencia.

Para finalizar se detallan las siguientes acciones positivas y negativas de estos campos:

- **Importancia en el diseño de bases de datos**

La correcta definición de campos relacionales y dependencias de existencia permite:

- Evitar registros inconsistentes.
- Mantener la coherencia entre datos.
- Reducir redundancia.
- Facilitar consultas complejas.
- Mejorar la organización del sistema.

- **Errores comunes en su implementación**

En el diseño de bases de datos, es frecuente encontrar errores relacionados con estos conceptos, como:

- Definir campos relacionales sin restricciones adecuadas.
- Permitir registros sin relación válida.
- Crear dependencias innecesarias.
- No validar la existencia de datos relacionados.

4. Lenguajes de sistemas gestores de bases de datos (DBMS)

Los lenguajes de sistemas gestores de bases de datos constituyen el conjunto de instrucciones que permiten interactuar con una base de datos, definiendo su estructura, manipulando la información, controlando el acceso y gestionando las transacciones. Estos lenguajes son fundamentales para el funcionamiento de los sistemas de información, ya que permiten a los usuarios y aplicaciones comunicarse con el sistema gestor de bases de datos (DBMS) de manera estructurada y eficiente.

En el contexto del modelo relacional, el lenguaje más utilizado es SQL (Structured Query Language), el cual se ha consolidado como el estándar para la gestión de bases de datos. A través de SQL, es posible crear, consultar, modificar y controlar los datos, garantizando su integridad y consistencia.

4.1. Lenguaje de definición de datos (DDL)

El lenguaje de definición de datos, conocido como DDL (Data Definition Language), es el conjunto de instrucciones utilizado en los sistemas gestores de bases de datos para definir, modificar y eliminar la estructura de los datos. Este lenguaje constituye la base sobre la cual se construyen las bases de datos, ya que permite establecer las tablas, atributos, relaciones y restricciones necesarias para organizar la información.

En el contexto del modelo relacional, el DDL permite crear las estructuras que soportan el almacenamiento de datos, asegurando que la base de datos cumpla con los requerimientos del sistema y garantice la integridad de la información.

El DDL cumple un papel fundamental en la creación y mantenimiento de la base de datos, permitiendo definir su arquitectura y garantizar su correcto funcionamiento, basado en las siguientes funciones principales:

- Crear estructuras de datos como tablas, índices y esquemas.
- Modificar estructuras existentes.
- Eliminar objetos de la base de datos.
- Definir restricciones de integridad.
- Establecer relaciones entre tablas.

A. Comandos principales del DDL

El DDL está compuesto por un conjunto de comandos que permiten gestionar la estructura de la base de datos. Estos comandos son fundamentales para la creación y administración de los objetos dentro del sistema. La siguiente es su explicación y ejemplificación:

- **CREATE**

El comando CREATE permite crear nuevos objetos dentro de la base de datos, como tablas, bases de datos, vistas o índices.

```
CREATE TABLE Estudiante (  
  
    ID_Estudiante INT PRIMARY KEY,  
  
    Nombre VARCHAR(50),  
  
    Edad INT  
  
);
```

- **ALTER**

El comando ALTER permite modificar la estructura de una tabla existente, ya sea agregando, eliminando o modificando columnas.

```
ALTER TABLE Estudiante
```

```
ADD Correo VARCHAR(100);
```

- **DROP**

El comando DROP se utiliza para eliminar completamente un objeto de la base de datos, como una tabla o una base de datos.

```
DROP TABLE Estudiante;
```

- **TRUNCATE**

El comando TRUNCATE elimina todos los registros de una tabla sin eliminar su estructura.

```
TRUNCATE TABLE Estudiante;
```

B. Restricciones con DDL

El DDL permite establecer restricciones que garantizan la integridad de los datos dentro de la base de datos. Estas restricciones controlan qué tipo de datos pueden almacenarse y cómo se relacionan entre sí. Los siguientes son los tipos existentes:

- **PRIMARY KEY:** identifica registros únicos.
- **FOREIGN KEY:** establece relaciones entre tablas.
- **NOT NULL:** evita valores nulos.
- **UNIQUE:** evita duplicados.

- CHECK: valida condiciones específicas.

```
CREATE TABLE Pedido (  
  
    ID_Pedido INT PRIMARY KEY,  
  
    ID_Cliente INT,  
  
    Total DECIMAL(10,2),  
  
    FOREIGN KEY (ID_Cliente) REFERENCES Cliente(ID_Cliente)  
  
);
```

Para entender de mejor manera el tema de las restricciones, se detalla el siguiente esquema explicativo:

- **EJEMPLO - CREACIÓN DE TABLAS CON RESTRICCIONES**

```
CREATE TABLE Empleados  
  
(  
  
    EmpleadoID int IDENTITY(1,1) NOT NULL,  
  
    Nombre nvarchar(50) NOT NULL CONSTRAINT UQ_Empleados_Nombre  
    UNIQUE,  
  
    Email nvarchar(100) NOT NULL,  
  
    DepartamentoID int NOT NULL,  
  
    FechaContratacion date NOT NULL,  
  
    Salario decimal(12,2) NOT NULL,
```

```
Estado bit NOT NULL CONSTRAINT DF_Empleados_Estado DEFAULT(1)

CONSTRAINT PK_Empleados PRIMARY KEY CLUSTERED (EmpleadoID),

CONSTRAINT FK_Empleados_Departamentos FOREIGN KEY (DepartamentoID)

REFERENCES Departamentos(DepartamentoID) ON UPDATE CASCADE,

CONSTRAINT CK_Empleados_Salario CHECK (Salario > 0),

CONSTRAINT CK_Empleados_Email CHECK (Email LIKE '%@_._%')

)
```

GO

```
CREATE TABLE Departamentos
```

```
( DepartamentoID int IDENTITY(1,1) NOT NULL, Nombre nvarchar(50) NOT
NULL CONSTRAINT UQ_Departamentos_Nombre UNIQUE, Ubicacion nvarchar(100)
NOT NULL, FechaCreacion date NOT NULL CONSTRAINT DF_Departamentos_Fecha
DEFAULT(GETDATE()),
```

```
CONSTRAINT PK_Departamentos PRIMARY KEY CLUSTERED (DepartamentoID)

)
```

GO

- **UNIQUE:** Restricción de valor único, permite que solo se permitan valores únicos sobre este campo.
- **DEFAULT:** Restricción de predeterminado, permite que de no llenarse el campo se asigne de manera automática un valor.

- **PRIMARY KEY:** Restricción de llave primaria, garantiza que este campo no permita duplicados y será el identificador único de cada fila.
- **FOREIGN KEY:** Una restricción de llave foránea hace que este campo guarde relación con un campo de otra tabla, al que hace referencia y no permite ingresar un valor si no existe en el campo de la otra tabla.
- **CHECK:** Restricción de regla de validación, no permite ingresar un valor al campo si este no cumple con la condición especificada.
- **DEFAULT (GETDATE()):** Asigna la fecha actual automáticamente si no se proporciona un valor.

Importancia del DDL en el diseño de bases de datos y su aplicación en entornos reales

El DDL es esencial en el proceso de diseño, ya que permite materializar el modelo lógico en una estructura implementada dentro del DBMS. A través de este lenguaje, se definen las tablas, relaciones y restricciones que garantizan el correcto funcionamiento del sistema.

El DDL es utilizado en la creación de bases de datos en sistemas empresariales, plataformas web, sistemas académicos y aplicaciones financieras. Su correcta aplicación permite construir estructuras sólidas que soportan grandes volúmenes de información.

4.2. Lenguaje de manipulación de datos (DML)

El lenguaje de manipulación de datos, conocido como DML (Data Manipulation Language), es el conjunto de instrucciones que permite gestionar la información almacenada en una base de datos. A través de este lenguaje, es posible realizar

operaciones fundamentales como la inserción, consulta, actualización y eliminación de datos.

El DML cumple un papel clave en la manipulación de la información dentro del sistema:

- Insertar nuevos registros en las tablas.
- Consultar datos almacenados.
- Modificar información existente.
- Eliminar registros cuando sea necesario.

A. Comandos principales del DML

A continuación, se presentan los principales comandos del DML, utilizados para ejecutar las operaciones básicas de gestión de datos dentro de una base de datos, junto con ejemplos que ilustran su uso:

- **INSERT:** permite agregar nuevos registros a una tabla.

```
INSERT INTO Cliente (ID, Nombre)
```

```
VALUES (1, 'Juan');
```

- **SELECT:** permite consultar información almacenada en una o varias tablas.

```
SELECT * FROM Cliente;
```

- **UPDATE:** permite modificar datos existentes dentro de una tabla.

```
UPDATE Cliente
```

```
SET Nombre = 'Carlos'
```

```
WHERE ID = 1;
```

- **DELETE:** permite eliminar registros de una tabla.

DELETE FROM Cliente

WHERE ID = 1;

4.3. Lenguaje de control de datos (DCL)

El Lenguaje de control de datos, conocido como DCL (Data Control Language), es el conjunto de instrucciones utilizadas para gestionar la seguridad y el acceso a la información dentro de un sistema gestor de bases de datos. Este lenguaje permite definir qué usuarios pueden acceder a la base de datos y qué tipo de operaciones pueden realizar sobre ella.

El DCL permite administrar los permisos y privilegios dentro del sistema para:

- Asignar permisos a usuarios o roles.
- Restringir accesos a determinadas operaciones.
- Controlar la seguridad de la base de datos.
- Garantizar la confidencialidad de la información.

A. Comandos principales del DCL

A continuación, se presentan los comandos de este tipo de lenguaje y un ejemplo puntual, los cuales permiten gestionar los permisos y niveles de acceso de los usuarios dentro de la base de datos:

- **GRANT:** permite otorgar permisos a un usuario o rol para realizar operaciones sobre la base de datos.

GRANT SELECT, INSERT ON Cliente TO usuario;

- **REVOKE:** permite retirar permisos previamente otorgados a un usuario o rol.

REVOKE INSERT ON Cliente FROM usuario;

El DCL es esencial en entornos donde múltiples usuarios acceden a la base de datos, ya que permite establecer niveles de acceso diferenciados según el rol de cada usuario.

4.4. Lenguaje de definición de vistas (VDL)

El Lenguaje de definición de vistas, conocido como VDL (View Definition Language), es el conjunto de instrucciones utilizado para crear, modificar y eliminar vistas dentro de una base de datos. Una vista es una representación virtual de los datos que se obtiene a partir de una consulta, sin almacenar físicamente la información; es una tabla virtual que se construye a partir de una o varias tablas reales mediante una consulta SQL. Aunque se presenta como una tabla, no almacena datos propios, sino que muestra información derivada de otras estructuras.

El VDL permite gestionar las vistas dentro del sistema para:

- Crear vistas a partir de consultas.
- Modificar vistas existentes.
- Eliminar vistas cuando ya no se requieren.
- Simplificar consultas complejas.
- Restringir acceso a ciertos datos.

A. Comandos principales del VDL

A continuación, se presentan los comandos del lenguaje de definición de vistas y un ejemplo representativo, los cuales permiten crear, modificar y eliminar vistas para facilitar el acceso y la organización de la información almacenada en la base de datos:

- **CREATE VIEW:** permite crear una vista basada en una consulta.

```
CREATE VIEW Vista_Clientes AS
```

```
SELECT Nombre, Ciudad
```

```
FROM Cliente;
```

- **ALTER VIEW:** permite modificar una vista existente.

```
ALTER VIEW Vista_Clientes AS
```

```
SELECT Nombre
```

```
FROM Cliente;
```

- **DROP VIEW:** permite eliminar una vista de la base de datos.

```
DROP VIEW Vista_Clientes;
```

Para interpretar de manera precisa lo explicado en los anteriores lenguajes, se recomienda acceder al siguiente video:

Video 1. Lenguajes DBMS



[Enlace de reproducción del video](#)

Transcripción del video: Lenguajes DBMS

Los lenguajes de los sistemas gestores de bases de datos (DBMS) permiten interactuar con la base de datos de forma estructurada, segura y eficiente. A través de estos lenguajes, el sistema puede definir estructuras, manipular información, controlar accesos y gestionar vistas, teniendo la base de datos como elemento central.

El lenguaje de definición de datos (DDL) se encarga de establecer y mantener la estructura lógica de la base de datos. Mediante comandos como CREATE, ALTER, DROP y TRUNCATE, es posible crear tablas, modificar su estructura, eliminar objetos y gestionar el contenido estructural del sistema, garantizando la integridad de los datos desde su diseño.

El lenguaje de manipulación de datos (DML) permite trabajar directamente con la información almacenada. A través de comandos como INSERT, SELECT, UPDATE y DELETE, el sistema puede agregar, consultar, modificar y eliminar registros, facilitando el uso operativo y cotidiano de la base de datos.

Por su parte, el lenguaje de control de datos (DCL) está orientado a la seguridad y administración de permisos. Mediante los comandos GRANT y REVOKE, se controla el acceso de los usuarios, definiendo qué operaciones pueden realizar dentro del sistema.

Finalmente, el lenguaje de definición de vistas (VDL) permite gestionar vistas virtuales mediante comandos como CREATE VIEW, ALTER VIEW y DROP VIEW, facilitando la consulta de información y restringiendo el acceso a datos sensibles.

En conjunto, estos lenguajes aseguran una gestión integral, organizada y confiable de las bases de datos.

5. Bases de datos NoSQL y su diseño

Las bases de datos NoSQL (Not Only SQL) representan un enfoque moderno para el almacenamiento y gestión de información, diseñado para manejar grandes volúmenes de datos, alta escalabilidad y estructuras flexibles. A diferencia de las bases de datos relacionales, las NoSQL no se basan exclusivamente en tablas, sino que permiten almacenar información en diversos formatos como documentos, pares clave-valor, columnas o grafos.

Las bases de datos NoSQL se diferencian por su enfoque flexible y orientado a rendimiento, destacándose porque:

- No requieren esquemas rígidos.
- Permiten manejar grandes volúmenes de datos.
- Son altamente escalables (horizontalmente).
- Soportan datos no estructurados.
- Ofrecen alto rendimiento en sistemas distribuidos.

5.1. Conceptos, principios y terminología NoSQL

Las bases de datos NoSQL surgen como una alternativa al modelo relacional tradicional, orientadas a gestionar grandes volúmenes de datos en entornos donde la flexibilidad, el rendimiento y la escalabilidad son fundamentales. Este enfoque permite almacenar información no estructurada o semiestructurada, adaptándose a las necesidades de aplicaciones modernas y sistemas distribuidos.

A. Conceptos de NoSQL

Desde el punto de vista conceptual, las bases de datos NoSQL se diferencian del modelo relacional al no depender de esquemas rígidos ni de estructuras basadas exclusivamente en tablas. En su lugar, emplean modelos de almacenamiento más flexibles que permiten gestionar distintos tipos de información. Entre sus principales características se encuentran:

- Datos no estructurados o semiestructurados.
- Almacenamiento flexible sin esquema fijo.
- Distribución de datos en múltiples nodos.
- Escalabilidad horizontal.

B. Principios de NoSQL

El diseño de las bases de datos NoSQL se fundamenta en principios que priorizan el rendimiento y la disponibilidad del sistema frente a la rigidez estructural. Estos principios permiten un funcionamiento eficiente en entornos de alta demanda y sistemas distribuidos, incluyendo:

- **Escalabilidad horizontal**

Capacidad del sistema para distribuir y replicar los datos en múltiples servidores, permitiendo aumentar el rendimiento y la capacidad de almacenamiento mediante la incorporación de nuevos nodos sin afectar la operación del sistema.

- **Alta disponibilidad**

Garantiza el acceso continuo a la información, incluso cuando uno o varios componentes del sistema presentan fallos, asegurando que los servicios permanezcan operativos y accesibles para los usuarios.

- **Tolerancia a fallos**

Permite que el sistema continúe funcionando de manera adecuada aun cuando se produzcan errores en algunos nodos o componentes, gracias a mecanismos de redundancia y replicación de datos.

- **Flexibilidad de datos**

Facilita la adaptación del modelo de datos a diferentes estructuras y formatos de información, permitiendo almacenar y modificar datos sin necesidad de definir esquemas rígidos, lo que favorece la evolución de las aplicaciones.

Un principio teórico clave en los sistemas NoSQL es el teorema CAP, el cual establece que un sistema distribuido no puede garantizar simultáneamente la consistencia, la disponibilidad y la tolerancia a particiones.

C. Terminología NoSQL

Las bases de datos NoSQL utilizan una terminología específica asociada a su arquitectura distribuida y a su forma de gestionar los datos. Algunos de los términos más relevantes son:

- **Nodo:** servidor individual que forma parte del sistema distribuido.
- **Clúster:** conjunto de nodos que trabajan de manera coordinada.
- **Partición:** división de los datos para su distribución entre nodos.

- **Replicación:** copia de datos en varios nodos para garantizar disponibilidad.
- **Sharding:** técnica de particionado horizontal para distribuir grandes volúmenes de datos.

5.2. Modelos de bases de datos NoSQL

Las bases de datos NoSQL implementan distintos modelos de almacenamiento que definen la forma en que la información es organizada y gestionada dentro del sistema. Cada modelo responde a necesidades específicas de las aplicaciones, permitiendo optimizar el acceso a los datos, el rendimiento y la escalabilidad en entornos distribuidos.

Los modelos de bases de datos NoSQL se agrupan según la manera en que almacenan y estructuran la información. Esta clasificación facilita la selección del modelo más adecuado de acuerdo con el tipo de datos y los requerimientos del sistema. Al respecto, se debe tener en cuenta la siguiente clasificación:

Tabla 8. Modelos de bases de datos NoSQL

Modelo	Descripción	Uso principal
Clave-valor.	Almacena la información como pares clave-valor.	Caché, gestión de sesiones.
Documental.	Utiliza documentos estructurados tipo JSON o BSON.	Aplicaciones web.
Columnar.	Organiza los datos por columnas en lugar de filas.	Big data, análisis

Modelo	Descripción	Uso principal
Grafos.	Representa los datos mediante nodos y relaciones.	Redes sociales

El uso de estos modelos permite seleccionar la estructura más adecuada según el tipo de aplicación, optimizando el rendimiento y facilitando el manejo de grandes volúmenes de datos.

5.3. Tipos de bases de datos NoSQL

Las bases de datos NoSQL se clasifican en distintos tipos de acuerdo con la forma en que almacenan, organizan y gestionan la información. Esta clasificación permite seleccionar el modelo más adecuado según las características de los datos y los requerimientos de la aplicación, optimizando aspectos como el rendimiento, la escalabilidad y la flexibilidad del sistema.

Cada una de estas bases se explica a continuación y se ejemplifica para tener mayor claridad y utilización en acciones reales:

A. Bases de datos clave-valor

Almacenan la información mediante pares formados por una clave única y un valor asociado. Este modelo se caracteriza por su simplicidad y rapidez en el acceso a los datos, ya que la recuperación de la información se realiza directamente a través de la clave. Son ampliamente utilizadas en sistemas que requieren respuestas rápidas, como el manejo de sesiones de usuario, caché de datos y configuraciones temporales.

Un ejemplo representativo de este tipo es Redis, una base de datos utilizada principalmente para el manejo de caché y sesiones de usuario. Redis almacena la información como pares clave-valor en memoria, lo que permite accesos extremadamente rápidos y es ideal para aplicaciones que requieren alto rendimiento y baja latencia.

B. Bases de datos documentales

Gestionan la información en documentos estructurados, comúnmente en formatos como JSON o BSON. Cada documento puede contener datos complejos y estructuras anidadas, lo que permite una representación más flexible de la información. Este modelo es especialmente adecuado para aplicaciones web y sistemas donde los datos presentan variabilidad en su estructura.

Un ejemplo ampliamente utilizado es MongoDB, una base de datos documental que almacena la información en documentos con formato JSON/BSON. Este modelo es común en aplicaciones web modernas, ya que permite manejar estructuras de datos flexibles y cambiantes sin necesidad de esquemas rígidos.

C. Bases de datos en columna

Organizan la información por columnas en lugar de filas, lo que facilita la ejecución de consultas analíticas sobre grandes volúmenes de datos. Este enfoque permite un acceso eficiente a conjuntos específicos de información y es ampliamente utilizado en entornos de análisis de datos, inteligencia de negocios y procesamiento masivo de información.

Un ejemplo destacado es Apache Cassandra, una base de datos diseñada para gestionar grandes volúmenes de datos distribuidos en múltiples nodos. Cassandra es

utilizada en sistemas de big data y análisis masivo de información, donde se requiere alta disponibilidad y escalabilidad horizontal.

D. Bases de datos orientadas a grafos

Orientadas a grafos se enfocan en representar relaciones complejas entre los datos mediante nodos y enlaces. Este modelo permite analizar de manera eficiente las conexiones entre entidades, siendo ideal para aplicaciones donde las relaciones son fundamentales, como redes sociales, sistemas de recomendación y análisis de vínculos.

Un ejemplo representativo es Neo4j, una base de datos orientada a grafos que modela la información mediante nodos y relaciones. Este sistema es ideal para aplicaciones donde las conexiones entre los datos son fundamentales, como redes sociales, sistemas de recomendación y análisis de relaciones complejas.

- **Importancia de la clasificación NoSQL**

La clasificación de las bases de datos NoSQL facilita la selección de la tecnología más adecuada según el tipo de aplicación y los datos que se desean gestionar, contribuyendo a mejorar el rendimiento, la escalabilidad y la eficiencia general del sistema.

5.4. Arquitecturas y esquemas de datos

Las bases de datos NoSQL se caracterizan por implementar arquitecturas distribuidas y esquemas de datos flexibles que permiten gestionar grandes volúmenes de información de manera eficiente. Dentro de estos enfoques, las bases de datos en

columna y las orientadas a grafos representan dos modelos ampliamente utilizados en entornos de alto rendimiento y análisis de datos complejos.

Las bases de datos NoSQL suelen operar bajo arquitecturas distribuidas, en las cuales la información se almacena en múltiples servidores interconectados; además, se destacan por su:

- Distribución de datos en nodos.
- Escalabilidad horizontal.
- Alta disponibilidad.
- Tolerancia a fallos.

El diseño de la arquitectura y del esquema en bases de datos NoSQL es fundamental para garantizar el rendimiento, la escalabilidad y la eficiencia del sistema.

5.5. Diseño de bases de datos NoSQL

El diseño de bases de datos NoSQL se fundamenta en un enfoque orientado al uso y al rendimiento, donde la estructura de los datos se define a partir de la forma en que serán consultados. A diferencia del modelo relacional, en NoSQL no se prioriza la normalización, sino la optimización de las consultas y el acceso eficiente a la información mediante estructuras flexibles adaptadas a las necesidades de la aplicación.

A. Patrones de acceso

Describen la manera en que los datos serán consultados dentro del sistema y constituyen uno de los elementos más importantes en el diseño de bases de datos

NoSQL. A partir de estos patrones se define cómo se almacenará la información, qué datos se agrupan y cómo se optimizan las operaciones de lectura y escritura. Entre los aspectos más relevantes se encuentran:

- Frecuencia de las consultas realizadas.
- Tipos de filtros y criterios de búsqueda utilizados.
- Volumen de datos accedidos en cada operación.
- Relación entre operaciones de lectura y escritura.

Un diseño adecuado basado en patrones de acceso permite mejorar el rendimiento del sistema y reducir la complejidad de las consultas.

B. Uso de claves en NoSQL

Las claves en las bases de datos NoSQL cumplen un papel fundamental en la localización y acceso rápido a los datos. A diferencia del modelo relacional, donde las claves estructuran las relaciones entre tablas, en NoSQL las claves se utilizan principalmente para identificar y recuperar registros de forma eficiente. En este contexto, se emplean distintos tipos de claves, tales como:

- Claves únicas para identificar registros de manera directa.
- Claves compuestas diseñadas para responder a consultas específicas.
- Claves optimizadas de acuerdo con los patrones de acceso definidos.

El uso adecuado de las claves permite minimizar el tiempo de respuesta y optimizar el rendimiento del sistema.

C. Análisis de relaciones en NoSQL

En las bases de datos NoSQL, las relaciones entre los datos no se gestionan mediante claves foráneas como en el modelo relacional. En su lugar, se utilizan enfoques alternativos que permiten mayor flexibilidad en la representación de la información. Los métodos más comunes son:

- **Incrustación (embedding)**

Los datos relacionados se almacenan dentro de un mismo documento, reduciendo la necesidad de consultas adicionales.

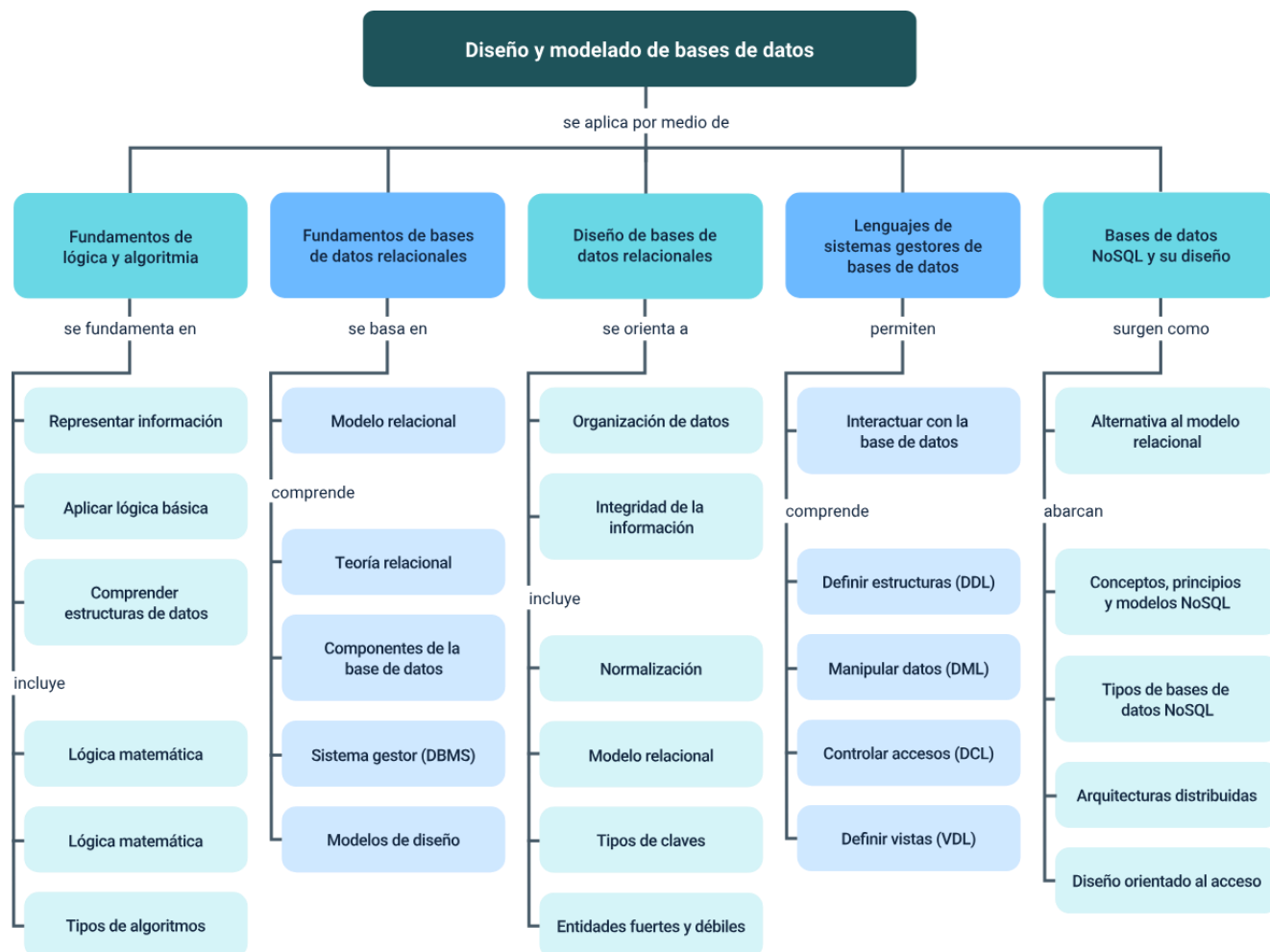
- **Referencia (linking)**

Los datos se relacionan mediante identificadores, permitiendo mantener estructuras más independientes.

La elección del enfoque adecuado depende del tipo de aplicación, del volumen de datos y de la frecuencia con que se consultan las relaciones.

Síntesis

El componente formativo desarrolla de manera progresiva los fundamentos de la lógica, la algoritmia y los sistemas de bases de datos, iniciando con los formatos de representación de la información y la clasificación de los distintos tipos de bases de datos, así como sus usos, diferencias y ventajas. Posteriormente, se abordan los principios de las bases de datos relacionales, incluyendo su teoría, componentes, sistemas gestores y modelos de diseño, profundizando en el proceso de normalización, el modelo relacional, los tipos de claves y las relaciones entre entidades. Asimismo, se estudian los lenguajes de los sistemas gestores de bases de datos, comprendiendo los lenguajes de definición, manipulación, control de datos y definición de vistas. Finalmente, el componente incorpora el estudio de las bases de datos NoSQL, sus conceptos, principios, modelos, arquitecturas y criterios de diseño, permitiendo comprender enfoques modernos de almacenamiento y gestión de información en entornos distribuidos, escalables y de alta demanda.



Glosario

Algoritmo: conjunto ordenado y finito de pasos que permite resolver un problema o realizar una tarea específica, garantizando un resultado determinado a partir de datos de entrada.

Base de datos relacional: sistema de almacenamiento de información organizado en tablas relacionadas entre sí mediante claves, que permite gestionar datos estructurados de forma consistente.

Lenguaje SQL: lenguaje estándar utilizado para gestionar bases de datos relacionales, permitiendo definir estructuras, manipular datos y controlar accesos mediante comandos como DDL, DML y DCL.

Normalización: proceso de organización de los datos en una base de datos relacional para reducir redundancias y mejorar la integridad, mediante la división en tablas relacionadas.

NoSQL: tipo de base de datos no relacional que permite almacenar datos no estructurados o semiestructurados, caracterizándose por su flexibilidad, escalabilidad y alto rendimiento en entornos distribuidos.

Patrón de acceso: forma en que los datos son consultados o utilizados dentro de un sistema, elemento clave en el diseño de bases de datos NoSQL para optimizar el rendimiento.

Sistema gestor de bases de datos (DBMS): software que permite crear, administrar y manipular bases de datos, garantizando la seguridad, integridad y acceso eficiente a la información.

Vista (view): estructura virtual basada en una consulta que permite visualizar datos de una o varias tablas sin almacenarlos físicamente, facilitando la seguridad y simplificación de consultas.

Referencias bibliográficas

Asaad, C. & Baïna, K. (2020). NoSQL Databases: Yearning for Disambiguation. arXiv.

Beaulieu, A. (2020). Learning SQL (3rd ed.). O'Reilly Media.

Codd, E. F. (1970). A Relational Model of Data for Large Shared Data Banks. Communications of the ACM.

Coronel, C. & Morris, S. (2018). Database Systems: Design, Implementation, & Management (13th ed.). Cengage Learning.

Date, C. J. (2004). An Introduction to Database Systems (8th ed.). Addison-Wesley.

Elmasri, R. & Navathe, S. (2016). Fundamentals of Database Systems (7th ed.). Pearson.

Sabharwal, N. & Gupta, S. (2015). Practical MongoDB. Apress.

Silberschatz, A., Korth, H., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill.

Créditos

Nombre	Cargo	Centro de Formación y Regional
Claudia Johanna Gómez Pérez	Profesional G06. Responsable Ecosistema de Recursos Educativos Digitales	Centro Agroturístico - Regional Santander
Diana Rocío Possos Beltrán	Responsable de línea de producción	Centro de Comercio y Servicios - Regional Tolima
Francisco José Vásquez Suárez	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Andrés Felipe Velandia Espitia	Evaluador instruccional	Centro de Comercio y Servicios - Regional Tolima
José Jaime Luis Tang Pinzón	Diseñador de contenidos digitales	Centro de Comercio y Servicios - Regional Tolima
Sebastian Trujillo Afanador	Desarrollador full stack	Centro de Comercio y Servicios - Regional Tolima
Ernesto Navarro Jaimes	Animador y productor audiovisual	Centro de Comercio y Servicios - Regional Tolima

Nombre	Cargo	Centro de Formación y Regional
Jorge Eduardo Rueda Peña	Evaluador de contenidos inclusivos y accesibles	Centro de Comercio y Servicios - Regional Tolima
Jorge Bustos Gómez	Validador y vinculator de recursos educativos digitales	Centro de Comercio y Servicios - Regional Tolima